

บทที่ 1

ความรู้เบื้องต้น

1.1 ความรู้เบื้องต้นระบบความต้องการวัสดุ (Material Requirement Planning)

เป็นความรู้ที่สำคัญที่สุดในการทำโครงการนี้ เพราะเป็นพื้นฐานการคิดในการทำงานในระบบ MRP ซึ่งข้อมูลและพื้นฐานความรู้การบริหารความต้องการวัสดุ จะมีอย่างละเอียดในปริิณยานิพนธ์ “ระบบคอมพิวเตอร์ช่วยในการวางแผนการผลิตของโรงงาน Material Requirement Planning” และในคู่มือฉบับนี้จะขออธิบายโดยสรุปดังนี้

1.1.1 วัตถุประสงค์

1. ออกแบบฐานข้อมูลด้านโรงงานอุตสาหกรรมการผลิต ตามหลักการการวางแผนความต้องการวัสดุ (Material Requirement Planning : MRP)
2. เขียนโปรแกรมประมวลผลข้อมูล ที่ใช้การบริหารตามหลักการการวางแผนความต้องการวัสดุเพื่อสามารถจำลองการทำงานของระบบ

1.1.2 ขอบเขตของโครงการ

ประกอบด้วยระบบย่อยดังต่อไปนี้

1. ระบบโครงสร้างผลิตภัณฑ์ (Bill of Material or Product Structure) เป็นระบบที่ใช้ฐานข้อมูลส่วนที่แสดงส่วนประกอบทั้งหมดที่ต้องใช้ในการผลิตสินค้า และระยะเวลาที่ใช้ในการผลิตสินค้านั้น
2. ระบบความต้องการวัสดุ สามารถคาดการณ์ว่าจะต้องมีการผลิตจำนวนเท่าใดในแต่ละช่วงเวลา
3. ระบบในการควบคุมสินค้าคงคลัง
4. ระบบสั่งซื้อสินค้าและวัตถุดิบที่ต้องใช้ในการผลิตให้ตรงตามเวลาที่ต้องการ
5. FG (Finish Good) ใ้ปิดการผลิต สามารถคิดต้นทุนการผลิต และส่งมอบสินค้าได้

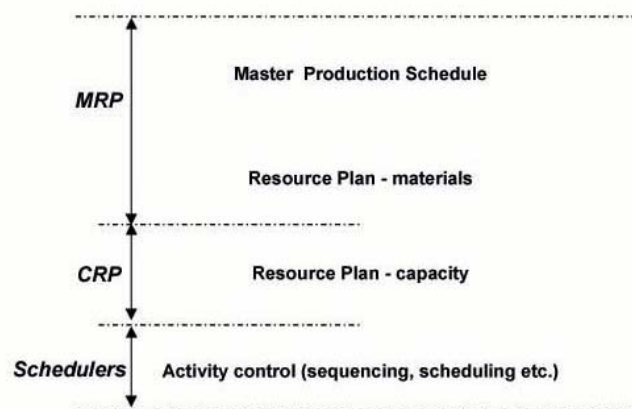
1.1.3 หลักการและทฤษฎี

Planning levels in the business

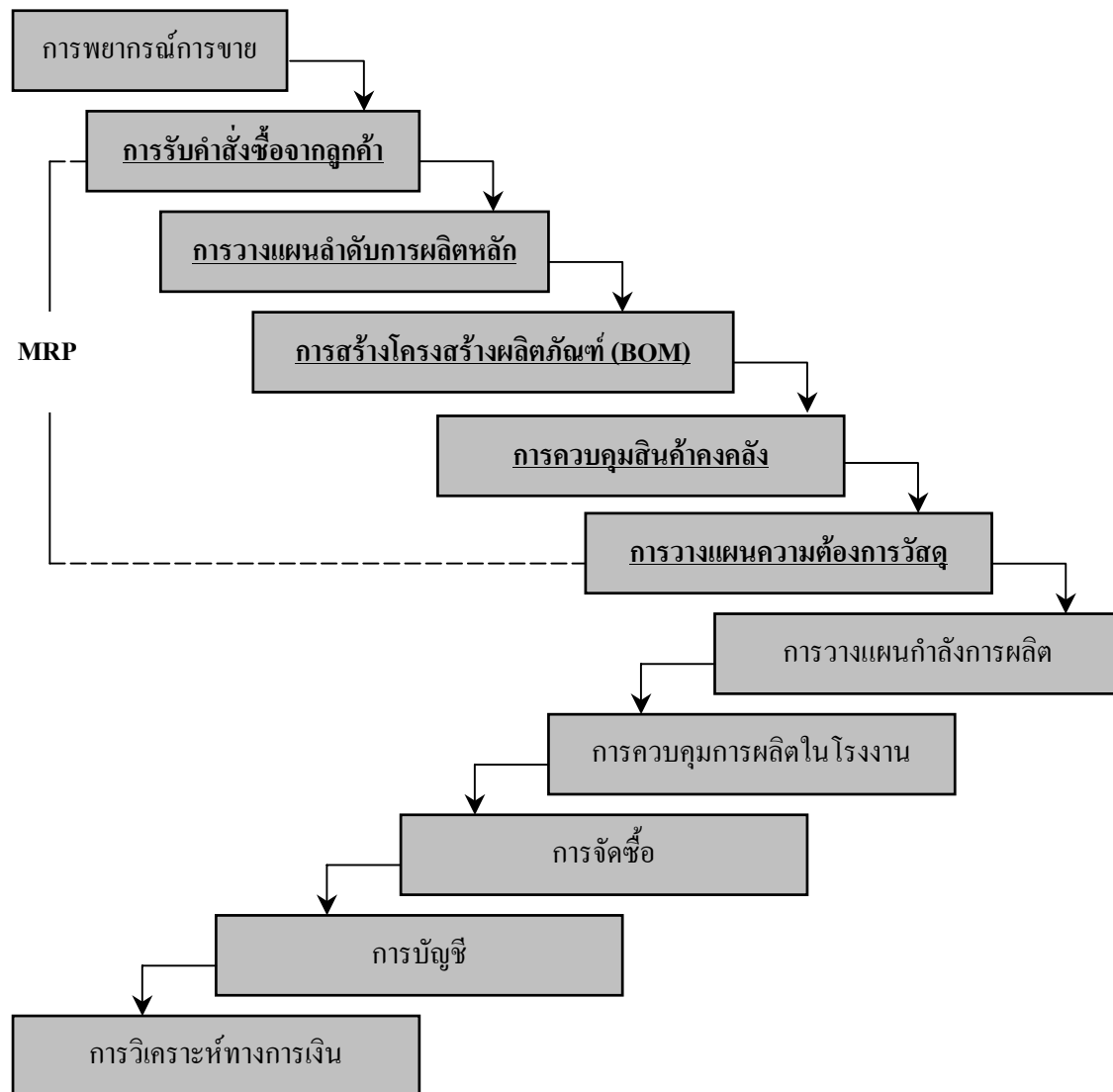


รูปที่ 1-1 แสดงการวางแผนจากบนลงล่าง

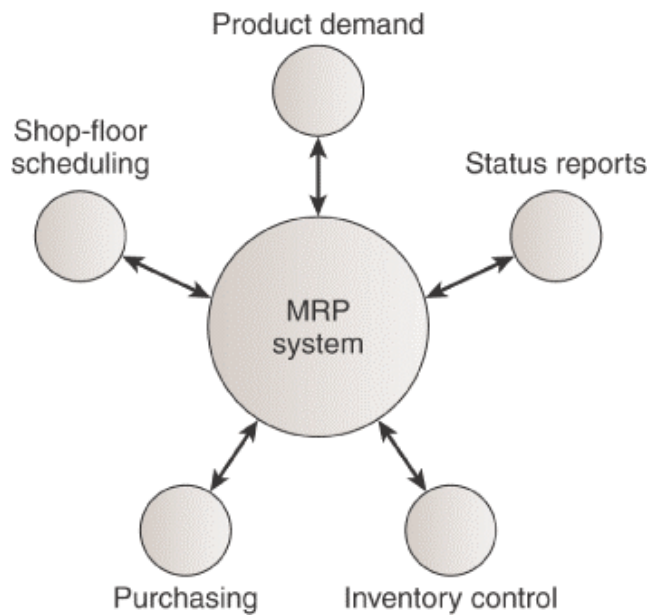
Planning levels - MRP/CRP/MRP2



รูปที่ 1-2 แสดงระดับชั้นการวางแผนกับระบบบริหารที่เกี่ยวข้อง



รูปที่ 1-3 แสดงการวางแผนในระบบการวางแผนความต้องการวัสดุแบบระบบปิด (Closed Loop MRP) กับระบบโดยรวมของการผลิต



รูปที่ 1-4 แสดงความสัมพันธ์ของระบบความต้องการวัสดุกับระบบที่เกี่ยวข้อง

MRP ตอบปัญหาเหล่านี้

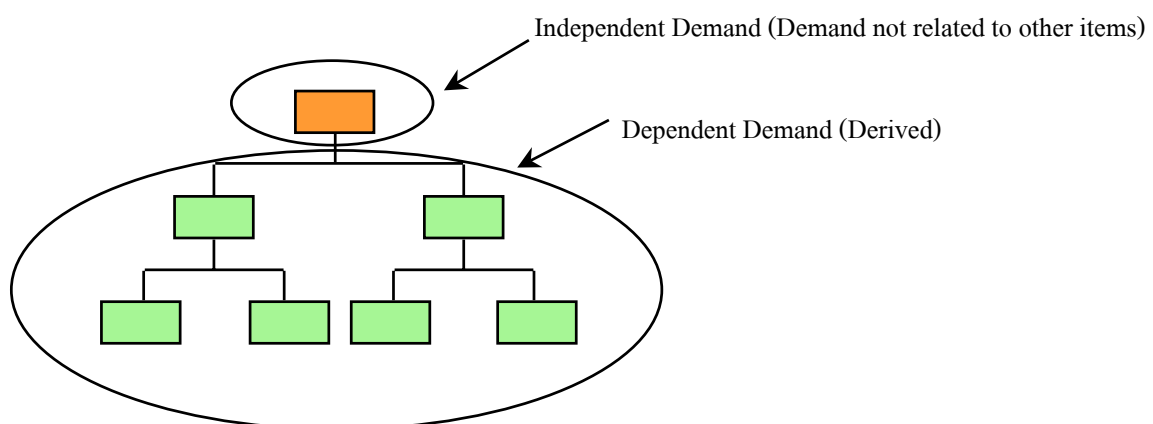
ต้องการอะไร (What is needed?)

ต้องการใช้มากเท่าใด (How much is needed?)

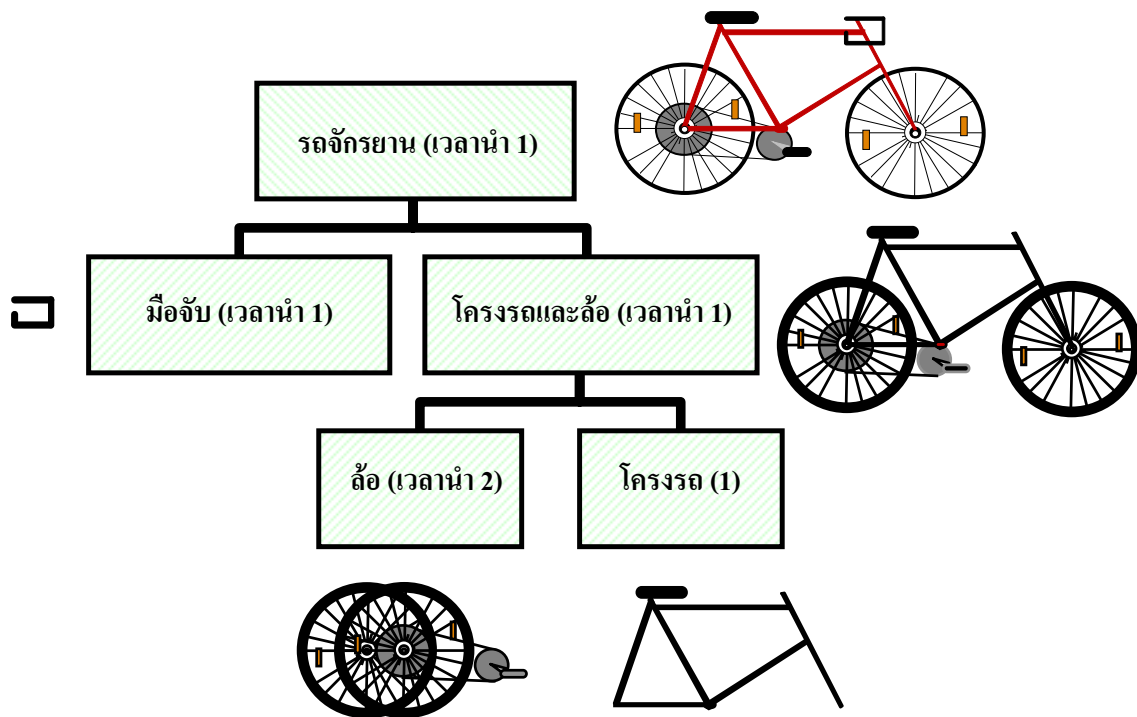
ต้องการใช้เมื่อใด (When is it needed?)

คุณสมบัติของ MRP

ชิ้นส่วนหรืออะไหล่ไม่จำเป็นต้องมีการพยากรณ์อุปสงค์ของแต่ละตัวแยกต่างหาก ถ้าทราบอุปสงค์ของสินค้าสำเร็จรูปที่ใช้ชิ้นส่วนก็จะทราบจำนวนชิ้นส่วนที่ต้องการ
อุปสงค์ของชิ้นส่วนจะไม่ต่อเนื่องสม่ำเสมอเช่นเดียวกับอุปสงค์ของสินค้าสำเร็จรูป

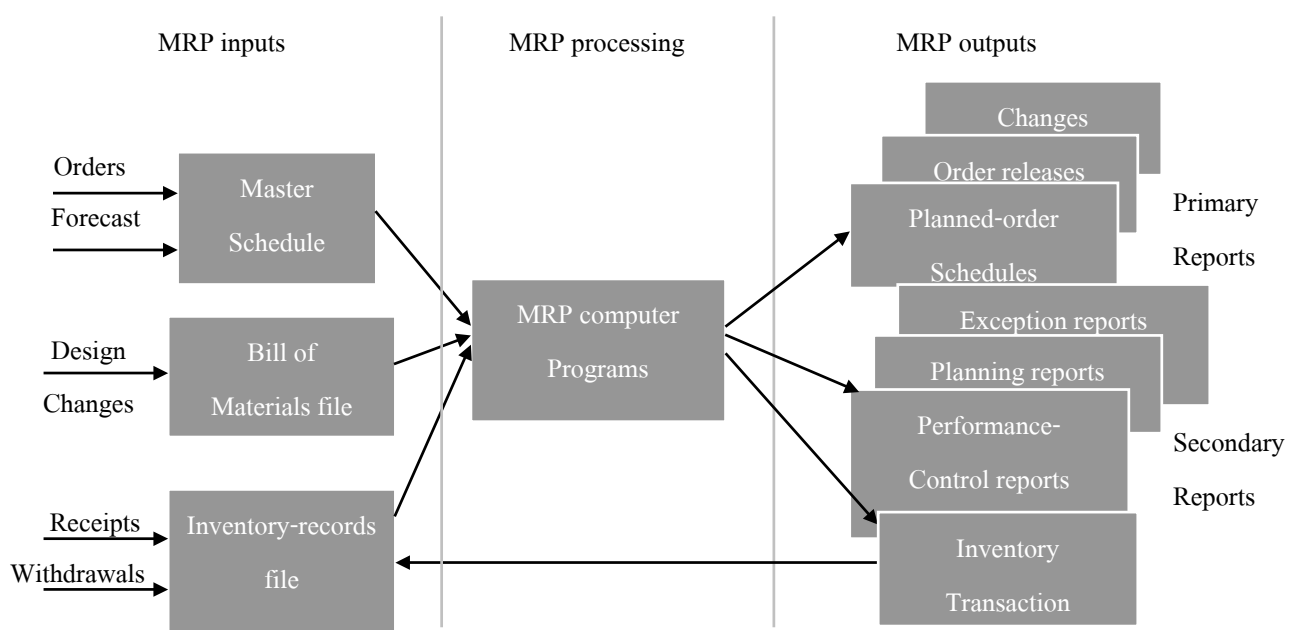


รูปที่ 1-5 โครงสร้างผลิตภัณฑ์ที่สัมพันธ์กับความต้องการ



รูปที่ 1-6 แสดงตัวอย่างโครงสร้างของผลิตภัณฑ์

อุปสงค์ทั้งสองจะเป็นไปพร้อมกัน โดยอุปสงค์แบบต่อเนื่องจะเป็นตัวเริ่มต้นของการทำงาน นั่นคือ เมื่อมีความต้องการสินค้าสำเร็จรูป (ความต้องการมืออยู่ตลอด หากไม่มีการยกเลิกความต้องการ) อุปสงค์จะต่อเนื่องตลอด แต่ส่วนที่ไม่ต่อเนื่องจะเป็นในส่วนขององค์ประกอบภายในสินค้าสำเร็จรูป ที่ไม่จำเป็นต้องมาใส่ประกอบชิ้นส่วนพร้อมกัน ยกตัวอย่างเช่น การผลิตโต๊ะ ที่ประกอบด้วยขาโต๊ะทั้งสี่ข้าง หากมีความต้องการโต๊ะจำนวน 100 ตัว จะต้องการ ขาโต๊ะ จำนวน 400 ขา เส้นกราฟตามรูป ทางด้านจำนวนสินค้าสำเร็จรูป จะเป็น 100 ตลอด ส่วนทางด้านจำนวนชิ้นส่วนจะต้องการ 400 เฉพาะบางช่วงเวลาเป็นต้น



รูปที่ 1-7 ส่วนประกอบของ MRP

สิ่งจำเป็นในระบบความต้องการวัสดุ (Requirements of MRP)

- ตารางการผลิตหลัก (Master production schedule)
 - จะผลิตอะไร เมื่อใด (When to make what)
- โครงสร้างผลิตภัณฑ์ (Bill of Materials)
 - ผลิตผลิตภัณฑ์นั้น ๆ อย่างไร (How to make the product)
- เวลามา (Lead times)
 - ผลิตส่วนต่าง ๆ ยาวนานแค่ไหน (How long it takes to get various components)
- ของคงคลังที่จัดหา (Inventory availability: Lot sizes)
- แผนการการรับและมีอยู่ของของคงคลัง (Scheduled receipts & on-hand inventory)

MRP inputs: แผนลำดับการผลิตหลัก

แผนลำดับการผลิตหลัก (Master Production Schedule: MPS) เป็นการกำหนดว่าจะผลิตสินค้าแบบใด ประเภทใด เป็นจำนวนเท่าไร และต้องการสินค้าเมื่อใด

MRP inputs: โครงสร้างผลิตภัณฑ์ (BOM : Bill of Material)

โครงสร้างผลิตภัณฑ์ (Product Structure: Bill of Material: BOM) เป็นการแสดงชิ้นส่วนประกอบของผลิตภัณฑ์ทั้งหมด

แบ่งย่อยลงเป็นลำดับชั้นตั้งแต่ชิ้นส่วนเบื้องต้น (Parents) จนถึงชิ้นส่วนสุดท้ายที่แยกย่อยต่อไปไม่ได้แล้ว (Child)

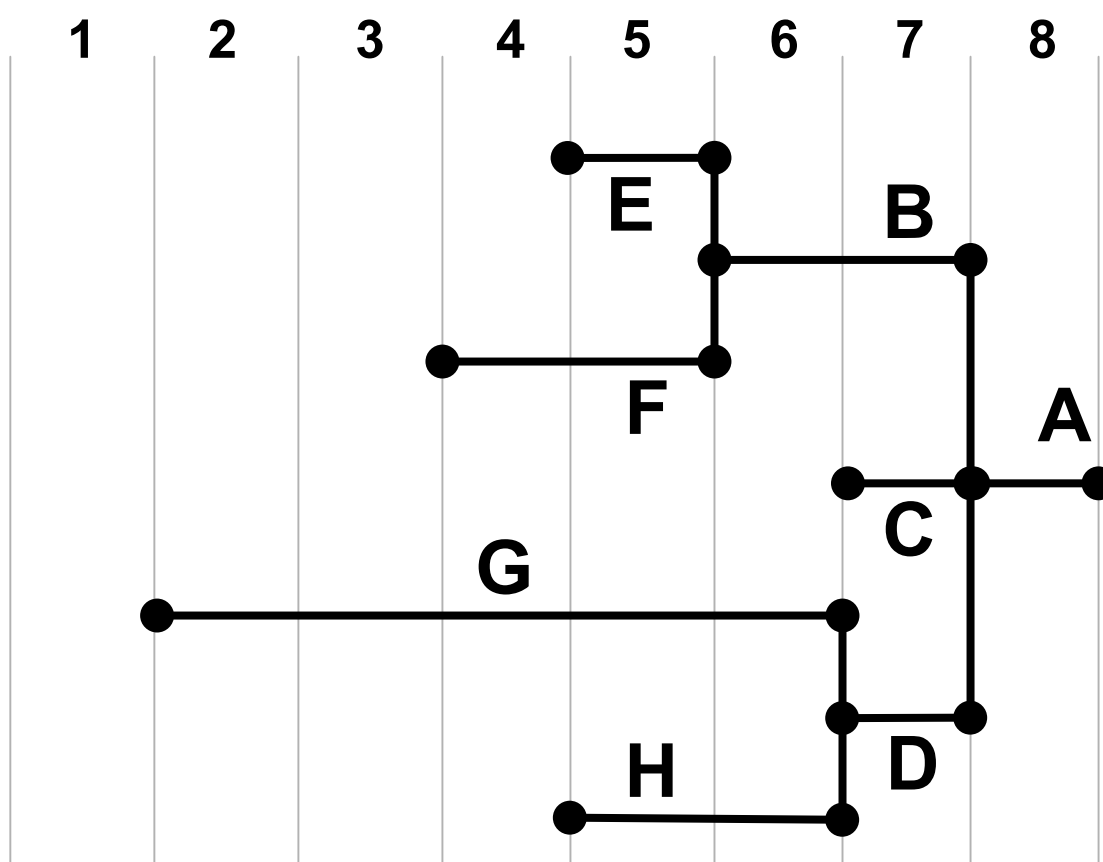
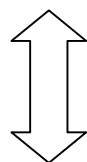
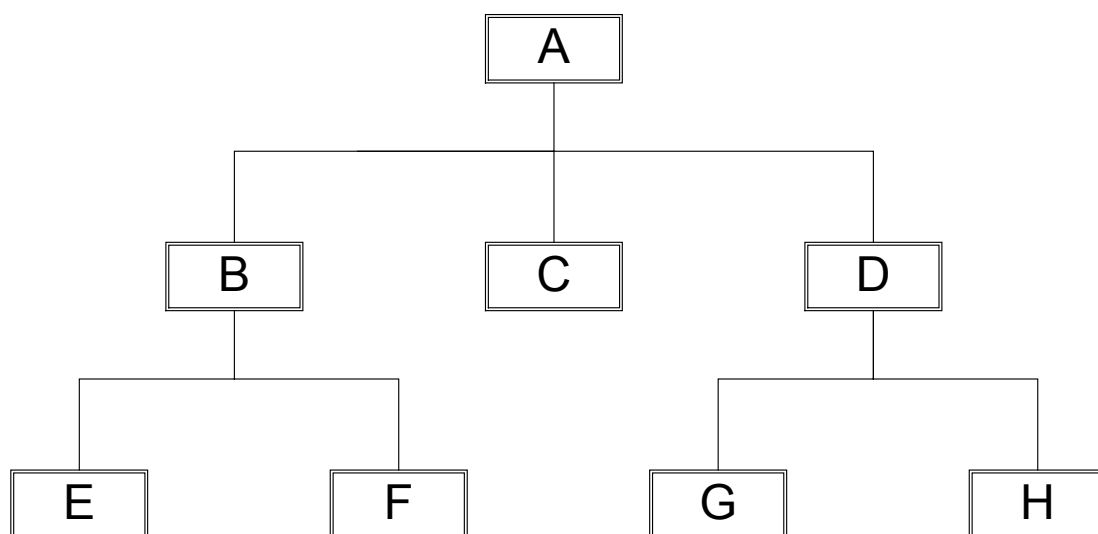
โครงสร้างผลิตภัณฑ์มีประโยชน์ในการคำนวณและวิเคราะห์ต้นทุน และแสดงรายละเอียดของงาน เรียกว่า Pick Lists

BOM ยังเป็นรายละเอียดในด้านจำนวนชิ้นส่วนต่างๆ ที่ใช้ในการผลิตผลิตภัณฑ์ 1 หน่วย และแสดงลำดับในการประกอบชิ้นส่วน

MRP inputs: แฟ้มข้อมูลสินค้าคงคลัง

แฟ้มข้อมูลสินค้าคงคลัง (Inventory Master File) เป็นข้อมูลรายละเอียดเกี่ยวกับสินค้าคงคลังแต่ละรายการทุกรายการ ในด้านปริมาณที่ยังคงเหลืออยู่, ปริมาณตั้งซื้อ, ขนาดของ Lot, ส่วนที่เพื่อขาดมือ, เวลา รอคอย และอัตราการใช้

ตัวอย่าง โครงสร้างผลิตภัณฑ์ และ แผนภูมิการผลิต



รูปที่ 1-8 แผนภูมิการผลิตพิจารณาพร้อมกับ Lead-times

MRP Processing

MRP คือการคิดย้อนกลับ

ระบบ MRP เป็นการคำนวณหาชิ้นส่วนทั้งหมดที่ต้องใช้ในช่วงเวลาต่างๆ ซึ่งมาจากโครงสร้างผลิตภัณฑ์และแผนลำดับการผลิตหลักของทุกผลิตภัณฑ์

ความต้องการรวม (Gross Requirements) คือ จำนวนของชิ้นส่วนที่ต้องการใช้ ในการประกอบเป็นชิ้นส่วนอื่น ในระดับสูงถัดขึ้นไปในโครงสร้างผลิตภัณฑ์ ตามจำนวนของสินค้าสำเร็จรูปที่แผนลำดับการผลิตหลักต้องการ ความต้องการรวมระดับล่างหาได้จากความต้องการรวมระดับบน

จำนวนที่จะได้รับ (Scheduled Receipts) คือ จำนวนชิ้นส่วนที่จะได้รับจากผู้ขายหรือรับจากแผนกผลิตชิ้นส่วนในช่วงเวลาต่างๆ

จำนวนที่ใช้ได้ (Available Balance or Projected on Hand) คือ จำนวนชิ้นส่วนที่สามารถนำไปใช้เพื่อการผลิตในช่วงเริ่มต้นของแต่ละช่วงเวลา คำนวณจาก

$$\text{จำนวนที่ใช้ได้} = \text{จำนวนที่จะได้รับ} + \text{สินค้าคงคลังยกมาจากช่วงเวลาก่อน}$$

ความต้องการสุทธิ (Net Requirements) คือ จำนวนสุทธิ (จริง) ของชิ้นส่วนที่ต้องการในแต่ละช่วง คำนวณจาก

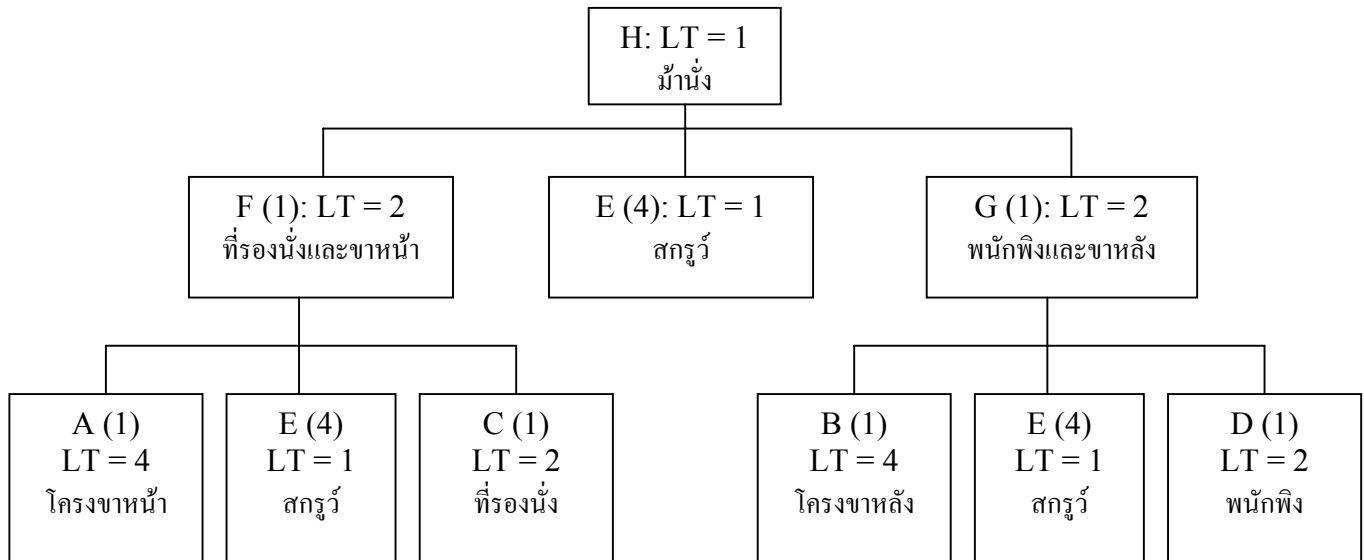
$$\text{ความต้องการสุทธิ} = \text{ความต้องการรวม} - \text{จำนวนที่ใช้ได้}$$

จำนวนรับตามแผน (Planned-Order Receipt) คือ จำนวนชิ้นส่วนที่จะได้รับตามแผนที่กำหนดเพื่อใช้ในการประกอบชิ้นส่วนอื่นในช่วงเริ่มต้นของช่วงเวลานั้น หากสั่งซื้อด้วยปริมาณคงที่ทุกครั้ง จำนวนรับตามแผนจะเท่ากับความต้องการสุทธิ

จำนวนสั่งตามแผน (Planned-Order Releases) คือ จำนวนชิ้นส่วนที่ต้องการสั่งซื้อหรือต้องสั่งผลิตในแต่ละช่วงเวลาตามกำหนด จำนวนสั่งตามแผนจะมีปริมาณเท่ากับจำนวนรับตามแผน โดยที่การรับจะต้องใช้เวลาเท่ากับเวลารอคอยภายหลังจากที่สั่งซื้อหรือสั่งผลิตแล้ว จำนวนสั่งตามแผนกลายเป็นความต้องการรวมของชิ้นส่วนที่อยู่ในระดับล่างของโครงสร้างผลิตภัณฑ์ต่อไป

ตัวอย่างการประมวลผลในระบบ MRP

สมมติ โรงงานจะผลิตม้านั่ง ต้องการผลิตม้านั่ง 1000 ตัว ภายใน 8 สัปดาห์หลังจากที่ได้รับคำสั่งซื้อ



โครงสร้างผลิตภัณฑ์ของการประกอบม้านั่ง

ผลิตโครงขาหน้า A		ประกอบที่รองนั่งและขาหน้า F		ประกอบพนักพิงและขาหลัง G		ประกอบม้านั่ง H	
		ที่รองนั่ง C	สั่งซื้อสกรู E		สั่งซื้อสกรู E		
		ผลิตโครงขาหลัง B					
		ผลิตพนักพิง D	สั่งซื้อสกรู E				

สัปดาห์ที่

ชิ้นส่วน (สินค้า)	จำนวนสินค้าคง คลัง	จำนวนสินค้าที่ สำรองไว้ (สินค้าเพื่อขาดมือ)	จำนวนที่จัดสรร แล้ว (จองแล้ว)	จำนวนที่จะได้รับ (สั่งซื้อไว้แล้ว)
H	200	100	0	0
F	104	60	40	0
G	400	60	120	0
A	100	40	60	100 ในต้นสัปดาห์ 3
B	300	40	60	0
C	120	40	60	0
D	104	40	60	100 ต้นสัปดาห์ 3
E	1000	600	300	0

รายละเอียด	สัปดาห์ที่	1	2	3	4	5	6	7	8
รหัส: H	ความต้องการรวม								1000
ระดับ: 0	จำนวนที่จะได้รับ								
สินค้าคงคลัง: 200	จำนวนที่ใช้ได้	100	100	100	100	100	100	100	0
สินค้าเพื่อขาดมือ: 100	ความต้องการสุทธิ								900
จำนวนที่จัดสรรแล้ว: 0	จำนวนรับตาม แผน								900
จำนวนที่จะได้รับ: 0	จำนวนสั่งตาม แผน							900	

รายละเอียด	สัปดาห์ที่	1	2	3	4	5	6	7	8
รหัส: F	ความต้องการรวม							900	
ระดับ: 1	จำนวนที่จะได้รับ								
สินค้าคงคลัง: 104	จำนวนที่ใช้ได้	4	4	4	4	4	4	0	
สินค้าเพื่อขาดมือ: 60	ความต้องการสุทธิ							896	
จำนวนที่จัดสรรแล้ว: 40	จำนวนรับตาม แผน							896	
จำนวนที่จะได้รับ: 0	จำนวนสั่งตาม แผน					896			

รายละเอียด	สัปดาห์ที่	1	2	3	4	5	6	7	8
รหัส: G	ความต้องการรวม							900	
ระดับ: 1	จำนวนที่จะได้รับ								
สินค้าคงคลัง: 400	จำนวนที่ใช้ได้	220	220	220	220	220	220	0	
สินค้าเพื่อขาดมือ: 60	ความต้องการสุทธิ							680	
จำนวนที่จัดสรรแล้ว: 120	จำนวนรับตาม แผน							680	
จำนวนที่จะได้รับ: 0	จำนวนส่งตาม แผน					680			

รายละเอียด	สัปดาห์ที่	1	2	3	4	5	6	7	8
รหัส: B	ความต้องการรวม					680			
ระดับ: 2	จำนวนที่จะได้รับ								
สินค้าคงคลัง: 300	จำนวนที่ใช้ได้	200	200	200	200	0			
สินค้าเพื่อขาดมือ: 40	ความต้องการสุทธิ					480			
จำนวนที่จัดสรรแล้ว: 60	จำนวนรับตาม แผน					480			
จำนวนที่จะได้รับ: 0	จำนวนส่งตาม แผน	480							

รายละเอียด	สัปดาห์ที่	1	2	3	4	5	6	7	8
รหัส: D	ความต้องการรวม					680			
ระดับ: 2	จำนวนที่จะได้รับ			100					
สินค้าคงคลัง: 104	จำนวนที่ใช้ได้	4	4	104	104	0			
สินค้าเพื่อขาดมือ: 40	ความต้องการสุทธิ					576			
จำนวนที่จัดสรรแล้ว: 60	จำนวนรับตาม แผน					576			
จำนวนที่จะได้รับ: 100	จำนวนส่งตาม แผน			576					

รายละเอียด	สัปดาห์ที่	1	2	3	4	5	6	7	8
รหัส: A	ความต้องการรวม					896			
ระดับ: 2	จำนวนที่จะได้รับ			100					
สินค้าคงคลัง: 100	จำนวนที่ใช้ได้	0	0	100	100	0			
สินค้าเพื่อขาดมือ: 40	ความต้องการสุทธิ					796			
จำนวนที่จัดสรรแล้ว: 60	จำนวนรับตาม แผน					796			
จำนวนที่จะได้รับ: 100	จำนวนส่งตาม แผน	796							

รายละเอียด	สัปดาห์ที่	1	2	3	4	5	6	7	8
รหัส: C	ความต้องการรวม					896			
ระดับ: 2	จำนวนที่จะได้รับ								
สินค้าคงคลัง: 120	จำนวนที่ใช้ได้	20	20	20	20	0			
สินค้าเพื่อขาดมือ: 40	ความต้องการสุทธิ					876			
จำนวนที่จัดสรรแล้ว: 60	จำนวนรับตาม แผน					876			
จำนวนที่จะได้รับ: 0	จำนวนส่งตาม แผน			876					

รายละเอียด	สัปดาห์ที่	1	2	3	4	5	6	7	8
รหัส: E	ความต้องการรวม					6304		3600	
ระดับ: 2	จำนวนที่จะได้รับ								
สินค้าคงคลัง: 1000	จำนวนที่ใช้ได้	100	100	100	100	0	0	0	
สินค้าเพื่อขาดมือ: 600	ความต้องการสุทธิ					6204		3600	
จำนวนที่จัดสรรแล้ว: 300	จำนวนรับตาม แผน					6201		3600	
จำนวนที่จะได้รับ: 0	จำนวนส่งตาม แผน				6204		3600		

MRP Outputs

1. รายงานการสั่งซื้อ (Purchase Orders) เป็นรายงานประจำ

- รายงานปริมาณชิ้นส่วนหรือวัตถุดิบที่จะต้องทำตามช่วงระยะเวลาต่างๆ
- แผนการสั่งซื้อหรือสั่งผลิต ซึ่งใช้ในการจัดทำใบสั่งซื้อหรือสั่งซื้อหรือสั่งผลิตของฝ่ายจัดซื้อ
- คำแนะนำในการซื้อ ใช้เมื่อเกิดการเปลี่ยนแปลงไปจากแผนที่กำหนดไว้จนต้องเปลี่ยนแปลงใบสั่งซื้อหรือสั่งผลิตที่ออกไปแล้ว

2. รายงานพิเศษ (Exception Report or Action Notices or Rescheduling Notices) เป็นรายงานเฉพาะเพื่อผู้บริหาร

- รายงานการควบคุมระบบ เช่น ข้อมูลจำนวนชิ้นส่วนที่นำเข้าสู่ระบบการผลิตไม่ได้ตามแผน ความคลาดเคลื่อนของเวลารอคอย
- แผนงานสินค้าคงคลัง ใช้พยากรณ์ความต้องการวัตถุดิบและชิ้นส่วน โดยมีสัญญาณสั่งซื้อและสั่งผลิตชิ้นส่วนและข้อมูลต่างๆ ประกอบ
- รายงานปัญหาพิเศษ เช่น ความล่าช้าของการสั่งซื้อชิ้นส่วน จำนวนชิ้นส่วนที่ไม่ได้มาตรฐาน

จุดประสงค์และหลักการ (Purposes and Philosophy)

จุดประสงค์ของระบบความต้องการวัสดุ (Purposes of MRP)

- สั่งสินค้าถูกต้องตาม ชิ้นส่วน เวลาและปริมาณ (Order the right part at the right time in the right quantity)
- รักษาเวลาที่กำหนดในการผลิต (Set and keep the due dates valid)

หลักการ (Philosophy)

เร่งงานเฉพาะเมื่อจำเป็น (Expedite only when necessary, de-expedite when appropriate)
(Better planning = better results)

คำศัพท์ที่ควรรู้

GR = Gross Requirement = ความต้องการขั้นต้น, ความต้องการรวม

SR = Schedule Receipts = จำนวนที่จะได้รับตามกำหนด

AI = Available Inventory = จำนวนของคงคลังที่สามารถนำไปใช้ได้

NR = Net Requirements = ความต้องการสุทธิ

POR = Planed Order Receipts = จำนวนรับตามแผน

PREL = Planed Order Releases = จำนวนสั่งตามแผน, แผนกำหนดสั่งของ

BI = Beginning Inventory = ของคงคลังต้นช่วงเวลา

1.2 การใช้งาน Visual Basic 6

1.2.1 การติดต่อกับฐานข้อมูล โดยใช้ Data Environment



รูปที่ 1-9 แสดง
DataEnvironment ที่ใช้

1.3 การใช้งาน Microsoft SQL 2000 Server

1.3.1 Store Procedure เพื่อเป็นคำสั่งที่ใช้งานที่ตัว Server เอง

บทที่ 2

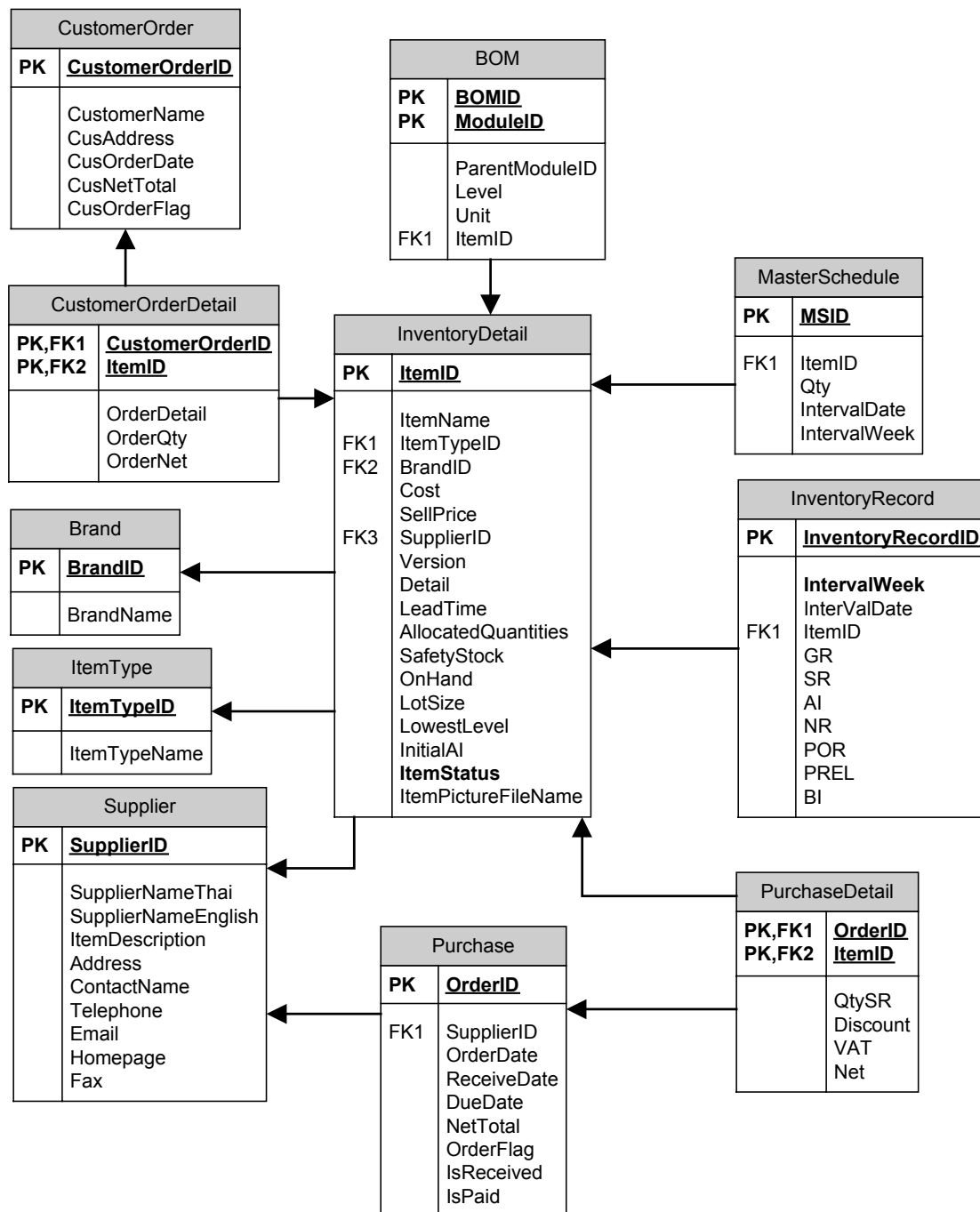
โค้ดและการทำงาน

2.1 การออกแบบ

ในการออกแบบระบบ MRP ได้ออกแบบโดยใช้สถาปัตยกรรมแบบ Client/Server แบบ 2-tier โดยส่วน database server ใช้ MS SQL 2000 SERVER เหตุผลที่เลือกใช้เพราะสามารถรองรับการใช้งานแบบ Multi user ได้ ในส่วนของ client ใช้ MS VISUAL BASIC ในการสร้าง

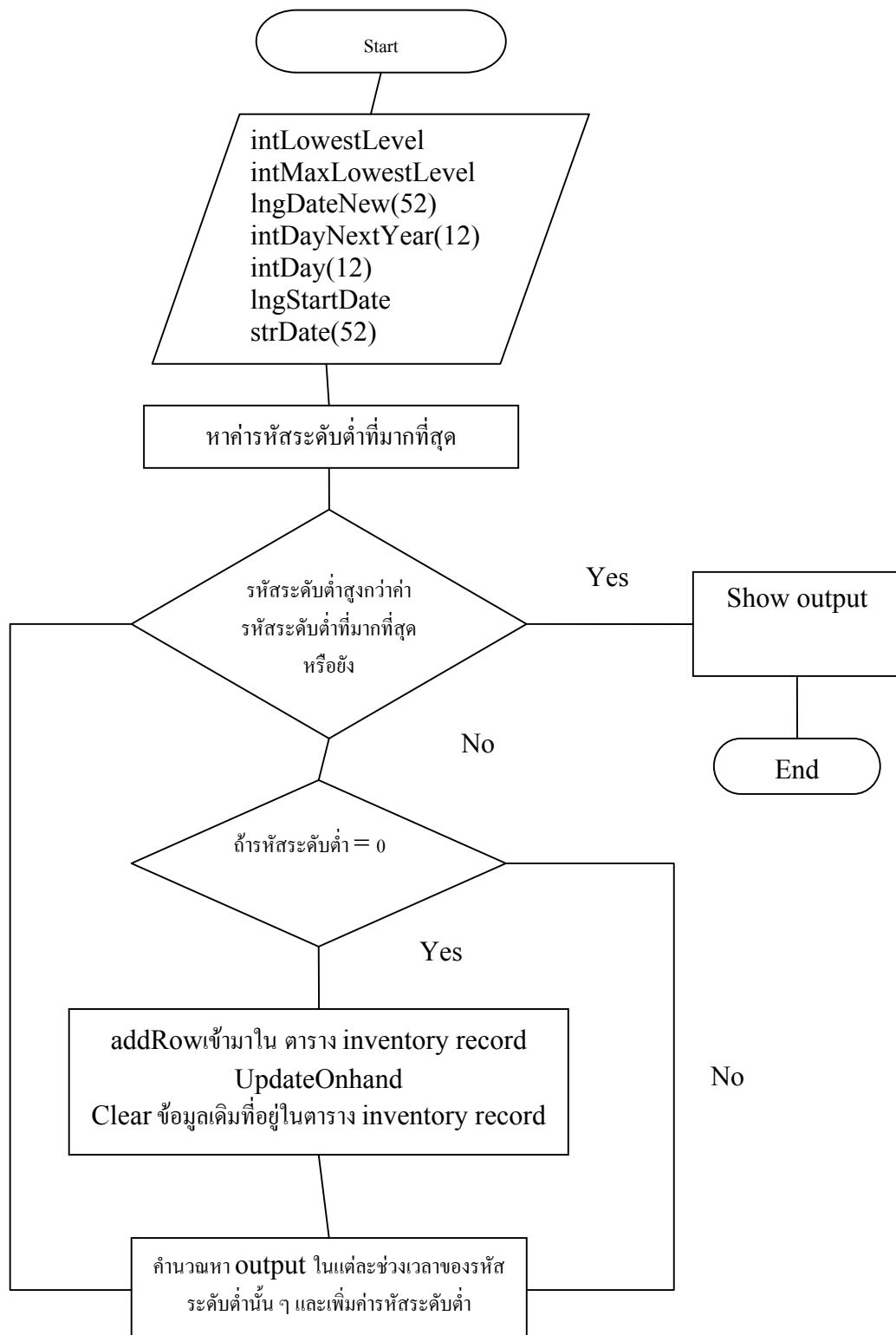
ในการออกแบบฐานข้อมูลใช้ฐานข้อมูลแบบ relational database โดยใช้ ER-Diagram ประกอบรายละเอียดของ Client program ประกอบด้วย

1. BOM manage form ในฟอร์มนี้ manager เท่านั้นที่สามารถเข้าใช้งานได้โดย manager สามารถ createBOM หรือ Load BOM มาแก้ไขได้ และในกรณีที่มิชชีนส่วนใหม่ที่ไม่มีในของคลังก็ต้องใส่รายละเอียดของชีนนั้นๆด้วย
2. form การเบิกจ่ายวัสดุ ในฟอร์มนี้ผู้ใช้ทั้ง manager และพนักงานสามารถสามารถใช้งานได้โดยเลือกชีนส่วนที่ทำการเบิกและพิมพ์ออกมาเป็นใบเบิกเพื่อเก็บไว้เป็นหลักฐานก่อนทำการเบิกจ่าย
3. form การสั่งวัสดุ ในฟอร์มนี้ผู้ใช้ทั้ง manager และพนักงานจะเป็นผู้เลือกชีนส่วนที่จะสั่งซื้อและพิมพ์ใบสั่งซื้อเพื่อทำการสั่งซื้อ
4. form การออกรายงาน ในฟอร์มนี้ผู้ใช้ทั้ง manager และพนักงานสามารถเลือกชนิดของรายงานต่างๆขึ้นมาดูได้และสามารถสั่งพิมพ์ได้ โดยมีชนิดของรายงาน เช่น แผนการใช้วัตถุดิบในแต่ละช่วงเวลา, ราคาต้นทุนของชีนส่วนแต่ละชีน, ราคาต้นทุนของสินค้าสำเร็จรูป เป็นต้น



รูปที่ 2-1 แสดงการออกแบบฐานข้อมูลโดยใช้ ER Schema

2.2 อธิบายโค้ดการทำงานของฟอร์ม Inventory Record



รูปที่ 2-2 แสดง Flow Chart การทำงานของฟอร์ม Inventory Record

ส่วนของตัวแปรที่ใช้ใน Form

Option Explicit

Dim intLowestLevel As Integer	เก็บระดับต่ำที่น้อยที่สุด
Dim intMaxLowestLevel As Integer	เก็บรหัสระดับต่ำที่มากที่สุด
Dim lngDateNew(52) As Long	เก็บวันที่ในช่วงของการนำมาคำนวณในทีนี้คือ 52 สัปดาห์
Dim intDayNextYear(12) As Integer	temp variable
Dim intDay(12) As Integer	temp variable
Dim lngStartDate As Long	เก็บวันที่เริ่มต้นการคำนวณ
Dim strDate(52) As String	เก็บวันที่เริ่มต้นการคำนวณในรูปแบบ string

Procedure ที่ทำงานตอน load form

Private Sub Form_Load()

On Error GoTo HaveError

Check connection

If DataEnvironment1.BOMConnection.State = adStateOpen Then

 DataEnvironment1.BOMConnection.Close

 DataEnvironment1.BOMConnection.Open

End If

If DataEnvironment1.BOMConnection.State = adStateClosed Then

 DataEnvironment1.BOMConnection.Open

End If

 DataEnvironment1.rsFindMaxLowestLevel.Open

หา รหัสระดับต่ำที่มากที่สุด

intMaxLowestLevel=DataEnvironment1.rsFindMaxLowestLevel.Fields("MaxLowestLevel").Value

DataEnvironment1.rsFindMaxLowestLevel.Close

คำนวณตั้งแต่รหัสระดับต่ำที่ 0 ถึงรหัสระดับต่ำสูงสุด

For intLowestLevel = 0 To intMaxLowestLevel

If intLowestLevel = 0 Then

initial

UpdateOnhand

lngStartDate = Year(Date) * 10000 + Month(Date) * 100 + Day(Date)

DataEnvironment1.ClearInventoryRecord lngStartDate

End If

LoadGR

CalSR_AI_NR_POR_PEI

CalPREL

CalGRNextLowestLevel

Next

ส่วนแสดงผลโดยใช้ MSHFlexGrid

DataEnvironment1.ShowItemID_ItemName lngDateNew(1)

MSHFlexGrid1.ColWidth(0) = 300

MSHFlexGrid1.ColWidth(4) = 800

MSHFlexGrid1.ColWidth(5) = 800

MSHFlexGrid1.ColWidth(0, 1) = 1000

MSHFlexGrid1.ColWidth(1, 1) = 1300

MSHFlexGrid1.ColWidth(2, 1) = 1100

MSHFlexGrid1.Width = 6000

MSHFlexGrid1.Height = 8800

Set MSHFlexGrid1.DataSource = DataEnvironment1.rsShowItemID_ItemName

DataEnvironment1.rsShowItemID_ItemName.Close

MSHFlexGrid1.CollapseAll

frmViewInventoryRecord.Caption = "Calculation Complete"

Exit Sub

HaveError:

MsgBox "Error Number: " & Err & " " & Error, vbOKOnly

End Sub

เมื่อมีการรับวัตถุดิบที่ส่งมาโปรแกรมจะทำการเพิ่มค่า SR ในช่วงเวลาที่จะถึงโดยทำงานจาก procedure นี้
ซึ่งจะทำงานในตอนเริ่มต้นการคำนวณ inventory record

Private Sub UpdateOnhand()

Dim i, intNoOfRecord As Integer

Dim intItemID() As Integer

Dim intPEI As Integer

Dim intAllocatedQuantities As Integer

Dim intSafetyStock As Integer

Dim intOnHand As Integer

Dim lngYear, lngMonth, lngDay, lngDate As Long

Dim blnStartDate As Boolean

lngYear = Year(Date)

lngMonth = Month(Date)

lngDay = Day(Date)

เป็นloopในการคำนวณหาวันที่ต้นสัปดาห์ถัดไป

Do While blnStartDate = False

If Weekday(DateSerial(lngYear, lngMonth, lngDay)) = 2 Then

blnStartDate = True

Else

If lngDay = 1 Then

If lngMonth = 1 Then

lngYear = lngYear - 1

lngMonth = 12

lngDay = 31

Else

```

        lngMonth = lngMonth - 1
        lngDay = intDay(lngMonth)
    End If
Else
    lngDay = lngDay - 1
End If
End If

Loop

lngDate = lngYear * 10000 + lngMonth * 100 + lngDay

```

เป็นการหา item ที่มีการรับวัตถุดิบเข้ามา

```

DataEnvironment1.rsSelectItemIDInventoryRecord.Open
intNoOfRecord = DataEnvironment1.rsSelectItemIDInventoryRecord.RecordCount
ReDim intItemID(intNoOfRecord) As Integer
DataEnvironment1.rsSelectItemIDInventoryRecord.MoveFirst

```

นำค่า itemID ที่รับวัตถุดิบเข้ามาเก็บไว้ในตัวแปล

```

For i = 1 To intNoOfRecord
    intItemID(i) = DataEnvironment1.rsSelectItemIDInventoryRecord.Fields("ItemID").Value
    DataEnvironment1.rsSelectItemIDInventoryRecord.MoveNext
Next

```

เป็น loop ในการ update ค่า

```

For i = 1 To intNoOfRecord
    DataEnvironment1.SelectAllocated_Safety intItemID(i)
    DataEnvironment1.SelectForUpdateOnhand intItemID(i), lngDate
    intAllocatedQuantities =
DataEnvironment1.rsSelectAllocated_Safety.Fields("AllocatedQuantities").Value
    intSafetyStock = DataEnvironment1.rsSelectAllocated_Safety.Fields("SafetyStock").Value
    If Not DataEnvironment1.rsSelectForUpdateOnhand.EOF Then
        intPEI = DataEnvironment1.rsSelectForUpdateOnhand.Fields("PEI").Value
        intOnHand = intPEI + intAllocatedQuantities + intSafetyStock
    End If
Next

```

```
DataEnvironment1.UpdateOnhand intOnHand, intItemID(i)
```

```
End If
```

```
DataEnvironment1.rsSelectAllocated_Safety.Close
```

```
DataEnvironment1.rsSelectForUpdateOnhand.Close
```

```
Next
```

```
End Sub
```

เป็น procedure ที่สร้างค่าเริ่มต้นของ inventoryRecord แล้บันทึกลง database

```
Private Sub initial()
```

ตัวแปรที่ใช้ใน procedure

```
Dim strDate(52) As String
```

```
Dim i, j, k As Integer
```

```
Dim intYear, intYearBuda As Integer
```

```
Dim strYear, strMonth, strDay As String
```

```
Dim intNumberOfItem, intItemID As Integer
```

```
Dim strItemName As String
```

```
Dim blnTmp1(52) As Boolean
```

```
Dim intNextYear As Integer
```

```
Dim lngYearNew, lngMonthNew, lngDayNew As Long
```

Loopในการหาว่าปีปัจจุบันมีเดือนละกี่วัน

For i = 1 To 12

 Select Case i

 Case 1

 intDay(1) = 31

 Case 2

 intYear = Year(Date)

 If intYear Mod 4 = 0 Then

 intDay(2) = 29

 Else

 intDay(2) = 28

 End If

 Case 3

 intDay(3) = 31

 Case 4

 intDay(4) = 30

 Case 5

 intDay(5) = 31

 Case 6

 intDay(6) = 30

 Case 7

 intDay(7) = 31

 Case 8

 intDay(8) = 31

 Case 9

 intDay(9) = 30

 Case 10

 intDay(10) = 31

 Case 11

 intDay(11) = 30

 Case 12

 intDay(12) = 31

 End Select

Next

Loop ในการหาว่าปีถัดไปมีเดือนละกี่วัน

For i = 1 To 12

 Select Case i

 Case 1

 intDayNextYear(1) = 31

 Case 2

 intNextYear = Year(Date) + 1

 If intNextYear Mod 4 = 0 Then

 intDayNextYear(2) = 29

 Else

 intDayNextYear(2) = 28

 End If

 Case 3

 intDayNextYear(3) = 31

 Case 4

 intDayNextYear(4) = 30

 Case 5

 intDayNextYear(5) = 31

 Case 6

 intDayNextYear(6) = 30

 Case 7

 intDayNextYear(7) = 31

 Case 8

 intDayNextYear(8) = 31

 Case 9

 intDayNextYear(9) = 30

 Case 10

 intDayNextYear(10) = 31

 Case 11

 intDayNextYear(11) = 30

 Case 12

 intDayNextYear(12) = 31

 End Select

Next


```
intYear = Year(Date)
lngYearNew = intYear * 10000
```

```
strYear = Str(intYear)
k = 1
j = Day(Date) + 1
```

loopในการคำนวณหาต้นช่วงเวลาทั้ง 52 ช่วงเวลา

```
Do While k <= 52
```

```
  For i = Month(Date) To 12
```

```
    lngMonthNew = i * 100
```

```
    strMonth = Str(i)
```

```
    Do While j <= intDay(i)
```

```
      If Weekday(DateSerial(intYear, i, j)) = 2 Then
```

```
        lngDayNew = j
```

```
        strDay = Str(j)
```

```
        strDate(k) = strDay & "/" & strMonth & "/" & strYear
```

```
        lngDateNew(k) = lngYearNew + lngMonthNew + lngDayNew
```

```
        If k < 52 Then
```

```
          k = k + 1
```

```
        Else
```

```
          GoTo nextStatement
```

```
        End If
```

```
      End If
```

```
      j = j + 1
```

```
    Loop
```

```
    j = 1
```

```
  Next
```

```
intYear = Year(Date) + 1
```

```

lngYearNew = intYear * 10000
strYear = Str(intYear)
For i = 1 To 12
    lngMonthNew = i * 100
    strMonth = Str(i)
    For j = 1 To intDayNextYear(i)
        If Weekday(DateSerial(intYear, i, j)) = 2 Then
            lngDayNew = j
            lngDateNew(k) = lngYearNew + lngMonthNew + lngDayNew
            strDay = Str(j)
            strDate(k) = strDay & "/" & strMonth & "/" & strYear

            If k < 52 Then
                k = k + 1
            Else
                GoTo nextStatement
            End If

        End If
    Next
Next
Loop
nextStatement:

DataEnvironment1.rsSelectItemID_ItemName.Open
DataEnvironment1.rsSelectItemID_ItemName.MoveFirst
intNumberOfItem = DataEnvironment1.rsSelectItemID_ItemName.RecordCount
DataEnvironment1.rsSelectItemID_ItemName.MoveFirst

For j = 1 To 52
    DataEnvironment1.SelectForCheckIntervalDate strDate(53 - j)
    If DataEnvironment1.rsSelectForCheckIntervalDate.EOF Then
        blnTmp1(53 - j) = True
    Else

```

```

        blnTmp1(53 - j) = False
    End If

    DataEnvironment1.rsSelectForCheckIntervalDate.Close

Next

For i = 1 To intNumberOfItem

    intItemID = DataEnvironment1.rsSelectItemID_ItemName.Fields("ItemID").Value
    strItemName = DataEnvironment1.rsSelectItemID_ItemName.Fields("ItemName").Value

    For j = 1 To 52

        If blnTmp1(j) Then
            DataEnvironment1.InsertInitialInventoryRecord    strDate(j),    intItemID,    strItemName,
lngDateNew(j)
        Else

            DataEnvironment1.CheckNewItemID intItemID, lngDateNew(j)

            If DataEnvironment1.rsCheckNewItemID.EOF Then
                DataEnvironment1.InsertInitialInventoryRecord    strDate(j),    intItemID,    strItemName,
lngDateNew(j)
            DataEnvironment1.rsCheckNewItemID.Close
            Else
                DataEnvironment1.rsCheckNewItemID.Close
            End If
        End If
    End If

Next

DataEnvironment1.rsSelectItemID_ItemName.MoveNext
DataEnvironment1.CheckItemID_ItemName intItemID

```

```

If Not DataEnvironment1.rsCheckItemID_ItemName.EOF Then

    If strItemName <> DataEnvironment1.rsCheckItemID_ItemName.Fields("ItemName").Value
Then
        DataEnvironment1.UpdateItemName strItemName, lngDateNew(1), intItemID
    End If

    DataEnvironment1.rsCheckItemID_ItemName.Close
End If

Next

DataEnvironment1.rsSelectItemID_ItemName.Close

End Sub

```

เป็น procedure ที่load ค่าGRจากตารางการผลิตหลักลง ในแต่ละ inventory record

```
Private Sub LoadGR() 'Load GR From Master Schedule
```

```

    If intLowestLevel = 0 Then

        DataEnvironment1.UpdateInventoryRecord1

    End If

End Sub

```

เป็น procedure ที่คำนวณค่า SR, AI, NR, POR, PEI

```

Private Sub CalSR_AI_NR_POR_PEI()

    Dim intIndex, i, j As Integer
    Dim intGR, intSR, intAI, intNR, intAIOld, intPOR, intAINew As Integer
    Dim intOnHand, intSafetyStock, intAllocatedQuantities As Integer
    Dim intGROld, intBIOld, intSROld, intBI As Integer

```

```

Dim intPEI, intPEIOld, intPEIFirst As Integer
Dim intLotSize, intTmp1, intIncrease, intFutureSR As Integer
Dim blnFutureSR As Boolean
Dim intPORForPEI As Integer
Dim intBIFuture As Integer

```

resetค่าต่างๆ

```

intIndex = 0
intGR = 0
intSR = 0
intAI = 0
intNR = 0
intAIOld = 0
intPOR = 0
intAINew = 0
intOnHand = 0
intSafetyStock = 0
intAllocatedQuantities = 0
intGROld = 0
intBIOld = 0
intSROld = 0
intBI = 0
intPEI = 0
intPEIOld = 0
intPEIFirst = 0
blnFutureSR = False

```

```

DataEnvironment1.selectItemID intLowestLevel
intIndex = DataEnvironment1.rsselectItemID.RecordCount

```

```

Dim intItemID() As Integer
ReDim intItemID(intIndex) As Integer
DataEnvironment1.rsselectItemID.MoveFirst

```

```

For i = 1 To intIndex
    intItemID(i) = DataEnvironment1.rsselectItemID.Fields("ItemID").Value
    DataEnvironment1.rsselectItemID.MoveNext
Next

For i = 1 To intIndex

    DataEnvironment1.selectGR_SR_AI intItemID(i), lngDateNew(1)
    DataEnvironment1.rsselectGR_SR_AI.MoveFirst

    For j = 1 To 52

        If j = 1 Then
            DataEnvironment1.SelectOnhand_SafetyStock intItemID(i)
            intOnHand = DataEnvironment1.rsSelectOnhand_SafetyStock.Fields("OnHand").Value
            intSafetyStock =
DataEnvironment1.rsSelectOnhand_SafetyStock.Fields("SafetyStock").Value
            intAllocatedQuantities =
DataEnvironment1.rsSelectOnhand_SafetyStock.Fields("AllocatedQuantities").Value
            intBI = intOnHand - intSafetyStock - intAllocatedQuantities
            If intBI < 0 Then
                intBI = 0
            End If
            intBIOld = intBI
            intGR = DataEnvironment1.rsselectGR_SR_AI.Fields("GR").Value
            If blnFutureSR = True Then
                intSR = intFutureSR
                blnFutureSR = False
            Else
                intSR = DataEnvironment1.rsselectGR_SR_AI.Fields("SR").Value
            End If
            intGROld = intGR
            intSROld = intSR
            intAINew = intBI + intSR

```

```
intNR = intGR - intAINew
```

```
If intNR < 0 Then
```

```
    intNR = 0
```

```
    intPOR = 0
```

```
Else
```

```
    intPOR = intNR
```

```
End If
```

```
DataEnvironment1.SelectLotSize intItemID(i)
```

```
intLotSize = DataEnvironment1.rsSelectLotSize.Fields("LotSize").Value
```

```
DataEnvironment1.rsSelectLotSize.Close
```

```
If intPOR > 0 And intPOR < intLotSize Then
```

```
    intPOR = intLotSize
```

```
End If
```

```
intPEIFirst = intOnHand - intSafetyStock - intAllocatedQuantities
```

```
intPEI = intPOR + intSR + intPEIFirst - intGR
```

```
intPEIOld = intPEI
```

```
DataEnvironment1.UpdateNR_POR_AI intNR, intPOR, intAINew, intBI, intPEI, intItemID(i),  
lngDateNew(j)
```

```
DataEnvironment1.rsselectGR_SR_AI.MoveNext
```

```
DataEnvironment1.rsSelectOnhand_SafetyStock.Close
```

```
Else
```

```
intGR = DataEnvironment1.rsselectGR_SR_AI.Fields("GR").Value
```

```
If blnFutureSR = True Then
```

```
    intSR = intFutureSR
```

```
    blnFutureSR = False
```

```
Else
```

```
    intSR = DataEnvironment1.rsselectGR_SR_AI.Fields("SR").Value
```

```

End If

intBI = intBIFuture

If intBI < 0 Then
    intBI = 0
End If

intAINew = intBI + intSR
intNR = intGR - intAINew

If intNR < 0 Then
    intNR = 0
    intPOR = 0
Else
    intPOR = intNR
End If

DataEnvironment1.SelectLotSize intItemID(i)
intLotSize = DataEnvironment1.rsSelectLotSize.Fields("LotSize").Value
DataEnvironment1.rsSelectLotSize.Close
If intPOR > 0 And intPOR < intLotSize Then

    intPOR = intLotSize

End If

intPEI = intPOR + intSR + intPEIOld - intGR
intPEIOld = intPEI
intBIFuture = intPEI

DataEnvironment1.UpdateNR_POR_AI intNR, intPOR, intAINew, intBI, intPEI, intItemID(i),
lngDateNew(j)

DataEnvironment1.rsselectGR_SR_AI.MoveNext
intBIOld = intBI
intGROld = intGR
intSROld = intSR

```


End If

Next

DataEnvironment1.rsselectGR_SR_AI.Close

Next

DataEnvironment1.rsselectItemID.Close

End Sub

เป็น procedure ที่คำนวณหาค่า PREL

Private Sub CalPREL()

Dim intIndex, i, j, intLotSize As Integer

Dim intPOR, intPREL, intLeadTime, intCriteria As Integer

Dim strCriteria, userCriteria As String

intIndex = 0

intPOR = 0

intPREL = 0

intLeadTime = 0

intCriteria = 0

strCriteria = ""

userCriteria = ""

DataEnvironment1.selectItemID intLowestLevel

intIndex = DataEnvironment1.rsselectItemID.RecordCount

Dim intItemID() As Integer

ReDim intItemID(intIndex) As Integer

DataEnvironment1.rsselectItemID.MoveFirst

For i = 1 To intIndex

```

intItemID(i) = DataEnvironment1.rsselectItemID.Fields("ItemID").Value
DataEnvironment1.rsselectItemID.MoveNext
Next

For i = 1 To intIndex

    DataEnvironment1.SelectPOR intItemID(i), lngDateNew(1)
    DataEnvironment1.SelectLeadTime intItemID(i)
    intLeadTime = DataEnvironment1.rsSelectLeadTime.Fields("LeadTime").Value

    For j = 1 To 52 - intLeadTime

        intCriteria = intLeadTime + j
        strCriteria = Str(lngDateNew(intCriteria))
        userCriteria = "intDate like '" & strCriteria & "'"

        DataEnvironment1.rsSelectPOR.MoveFirst
        DataEnvironment1.rsSelectPOR.Find userCriteria, , adSearchForward

        If Not DataEnvironment1.rsSelectPOR.EOF Then

            intPREL = DataEnvironment1.rsSelectPOR.Fields("POR").Value
            DataEnvironment1.UpdatePREL intPREL, intItemID(i), lngDateNew(j)

        End If

    Next

    DataEnvironment1.rsSelectPOR.Close
    DataEnvironment1.rsSelectLeadTime.Close

Next

DataEnvironment1.rsselectItemID.Close
End Sub

```

เป็น procedure ที่คำนวณค่า GR ของ item ที่มีรหัสระดับต่ำมากกว่า 1

Private Sub CalGRNextLowestLevel() ' fill New GR OF Child Module

```

Dim intModuleID, intBOMID, i, j, intIndex As Integer
Dim intChildItemID, intChildUnit, intGRChild, intNumberOfChildren As Integer
Dim intPREL, intIncrease As Integer

intModuleID = 0
intBOMID = 0
intIndex = 0
intChildItemID = 0
intChildUnit = 0
intGRChild = 0
intNumberOfChildren = 0
intPREL = 0
intIncrease = 0

If intLowestLevel <> intMaxLowestLevel Then

    DataEnvironment1.selectItemID intLowestLevel
    intIndex = DataEnvironment1.rsselectItemID.RecordCount
    Dim intItemID() As Integer
    ReDim intItemID(intIndex) As Integer
    DataEnvironment1.rsselectItemID.MoveFirst

    For i = 1 To intIndex
        intItemID(i) = DataEnvironment1.rsselectItemID.Fields("ItemID").Value
        DataEnvironment1.rsselectItemID.MoveNext
    Next

    For i = 1 To intIndex
        DataEnvironment1.SelectBOMID_ModuleID intItemID(i)
        DataEnvironment1.rsSelectBOMID_ModuleID.MoveFirst

        intModuleID = DataEnvironment1.rsSelectBOMID_ModuleID.Fields("ModuleID").Value
        intBOMID = DataEnvironment1.rsSelectBOMID_ModuleID.Fields("BOMID").Value
    
```

```
DataEnvironment1.SelectItemID_UnitChildren intBOMID, intModuleID
```

```
If Not DataEnvironment1.rsSelectItemID_UnitChildren.EOF Then
```

```
    DataEnvironment1.rsSelectItemID_UnitChildren.MoveFirst
```

```
End If
```

```
Do While Not DataEnvironment1.rsSelectItemID_UnitChildren.EOF
```

```
    intChildUnit = DataEnvironment1.rsSelectItemID_UnitChildren.Fields("Unit").Value
```

```
    intChildItemID = DataEnvironment1.rsSelectItemID_UnitChildren.Fields("ItemID").Value
```

```
    For j = 1 To 52
```

```
        DataEnvironment1.SelectGRForUpdate intChildItemID, lngDateNew(j)
```

```
        intGRChild = DataEnvironment1.rsSelectGRForUpdate.Fields("GR").Value
```

```
        DataEnvironment1.SelectPREL intItemID(i), lngDateNew(j)
```

```
        intPREL = DataEnvironment1.rsSelectPREL.Fields("PREL").Value
```

```
        intIncrease = intChildUnit * intPREL
```

```
        intGRChild = intGRChild + intIncrease
```

```
        DataEnvironment1.UpdateGRChildren intGRChild, intChildItemID, lngDateNew(j)
```

```
        DataEnvironment1.rsSelectGRForUpdate.Close
```

```
        DataEnvironment1.rsSelectPREL.Close
```

```
    Next
```

```
    DataEnvironment1.rsSelectItemID_UnitChildren.MoveNext
```

```
Loop
```

```
DataEnvironment1.rsSelectItemID_UnitChildren.Close
```

```
DataEnvironment1.rsSelectBOMID_ModuleID.Close
```

```
Next
```

```
DataEnvironment1.rsselectItemID.Close
```

End If

End Sub

2.3 อธิบายโค้ดการทำงานของฟอร์ม Master Schedule

เป็น variable ที่ใช้ใน form

Option Explicit

Dim intLowestLevel As Integer

Dim intMaxLowestLevel As Integer

Dim strDate(52) As String

Dim lngDateNew(52) As Long

Dim intDay(12), intDayNextYear(12) As Integer

Dim lngDateMin, lngDateMax As Long

เป็นส่วนที่ทำงานตอบload form

Private Sub Form_Load()

FindDate

AddMasterSchedule

ShowData

End Sub

เป็นprocedure ที่คำนวณหาต้นสัปดาห์ที่จะถึงเพื่อนำมาคำนวณ

Private Sub FindDate()

Dim i, j, k As Integer

Dim intYear, intYearBuda As Integer

Dim strYear, strMonth, strDay As String

```
Dim intNumberOfItem, intItemID As Integer
Dim strItemName As String
Dim lngMonthNew, lngDayNew, lngYearNew As Long
Dim intNextYear As Integer
```

```
For i = 1 To 12
```

```
  Select Case i
```

```
    Case 1
```

```
      intDay(1) = 31
```

```
    Case 2
```

```
      intYear = Year(Date)
```

```
      If intYear Mod 4 = 0 Then
```

```
        intDay(2) = 29
```

```
      Else
```

```
        intDay(2) = 28
```

```
      End If
```

```
    Case 3
```

```
      intDay(3) = 31
```

```
    Case 4
```

```
      intDay(4) = 30
```

```
    Case 5
```

```
      intDay(5) = 31
```

```
    Case 6
```

```
      intDay(6) = 30
```

```
    Case 7
```

```
      intDay(7) = 31
```

```
    Case 8
```

```
      intDay(8) = 31
```

```
    Case 9
```

```
      intDay(9) = 30
```

```
    Case 10
```

```
      intDay(10) = 31
```

```
    Case 11
```

```
      intDay(11) = 30
```

```

    Case 12
        intDay(12) = 31
    End Select
Next

For i = 1 To 12
    Select Case i
        Case 1
            intDayNextYear(1) = 31
        Case 2
            intNextYear = Year(Date) + 1
            If intNextYear Mod 4 = 0 Then
                intDayNextYear(2) = 29
            Else
                intDayNextYear(2) = 28
            End If
        Case 3
            intDayNextYear(3) = 31
        Case 4
            intDayNextYear(4) = 30
        Case 5
            intDayNextYear(5) = 31
        Case 6
            intDayNextYear(6) = 30
        Case 7
            intDayNextYear(7) = 31
        Case 8
            intDayNextYear(8) = 31
        Case 9
            intDayNextYear(9) = 30
        Case 10
            intDayNextYear(10) = 31
        Case 11
            intDayNextYear(11) = 30

```

Case 12

intDayNextYear(12) = 31

End Select

Next

intYear = Year(Date)

lngYearNew = intYear * 10000

strYear = Str(intYear)

k = 1

j = Day(Date)

Do While k <= 52

For i = Month(Date) To 12

lngMonthNew = i * 100

strMonth = Str(i)

Do While j <= intDay(i)

If Weekday(DateSerial(intYear, i, j)) = 2 Then

lngDayNew = j

strDay = Str(j)

strDate(k) = strDay & "/" & strMonth & "/" & strYear

lngDateNew(k) = lngYearNew + lngMonthNew + lngDayNew

If k < 52 Then

k = k + 1

Else

GoTo nextStatement

End If

End If

j = j + 1

Loop


```

    j = 1
Next

intYear = Year(Date) + 1
lngYearNew = intYear * 10000
strYear = Str(intYear)
For i = 1 To 12
    lngMonthNew = i * 100
    strMonth = Str(i)
    For j = 1 To intDayNextYear(i)
        If Weekday(DateSerial(intYear, i, j)) = 2 Then
            lngDayNew = j
            lngDateNew(k) = lngYearNew + lngMonthNew + lngDayNew
            strDay = Str(j)
            strDate(k) = strDay & "/" & strMonth & "/" & strYear

            If k < 52 Then
                k = k + 1
            Else
                GoTo nextStatement
            End If

        End If
    Next
Next
Loop

nextStatement:

End Sub

```

เป็น procedure ที่คำนวณ และupdate ตารางการผลิตหลัก

```
Private Sub AddMasterSchedule()
```

```
Dim i, intNoFinishGoods As Integer
```

```
Dim lngDay, lngMonth, lngYear, lngDate, lngTmp1, lngDayMin As Long
```

```
Dim AddsMasterSchedule() As MasterSchedule
```

```
Dim j, k, intCustomerOrderID, intNoCusOrderID, intIndex, intCountItemID, intCusItemID, intQty As Integer
```

```
Dim blnStartDate As Boolean
```

```
lngYear = Year(Date)
```

```
lngMonth = Month(Date)
```

```
lngDay = Day(Date)
```

```
lngDateMax = lngYear * 10000 + lngMonth * 100 + lngDay
```

```
Do While blnStartDate = False
```

```
    If Weekday(DateSerial(lngYear, lngMonth, lngDay)) = 2 Then
```

```
        blnStartDate = True
```

```
    Else
```

```
        If lngDay = 1 Then
```

```
            If lngMonth = 1 Then
```

```
                lngYear = lngYear - 1
```

```
                lngMonth = 12
```

```
                lngDay = 31
```

```
            Else
```

```
                lngMonth = lngMonth - 1
```

```
                lngDay = intDay(lngMonth)
```

```
            End If
```

```
        Else
```

```
            lngDay = lngDay - 1
```

```
        End If
```

```
    End If
```

Loop

lngDateMin = lngYear * 10000 + lngMonth * 100 + lngDay

DataEnvironment1.DeleteMasterSchedule lngDateMin

DataEnvironment1.SelectCusOrderID lngDateMin, lngDateMax

If DataEnvironment1.rsSelectCusOrderID.EOF Then

GoTo endSub

End If

DataEnvironment1.rsSelectAllFinishGoods.Open

intNoFinishGoods = DataEnvironment1.rsSelectAllFinishGoods.RecordCount

ReDim AddsMasterSchedule(intNoFinishGoods) As MasterSchedule

DataEnvironment1.rsSelectAllFinishGoods.MoveFirst

For i = 1 To intNoFinishGoods

AddsMasterSchedule(i).intItemID

=

DataEnvironment1.rsSelectAllFinishGoods.Fields("ItemID").Value

DataEnvironment1.SelectCriticalPath AddsMasterSchedule(i).intItemID

intIndex = DataEnvironment1.rsSelectCriticalPath.Fields("CriticalPath").Value + 1

AddsMasterSchedule(i).strIntervalDate = strDate(intIndex)

AddsMasterSchedule(i).lngDate = lngDateNew(intIndex)

DataEnvironment1.rsSelectAllFinishGoods.MoveNext

DataEnvironment1.rsSelectCriticalPath.Close

Next

intNoCusOrderID = DataEnvironment1.rsSelectCusOrderID.RecordCount

DataEnvironment1.rsSelectCusOrderID.MoveFirst

For i = 1 To intNoCusOrderID

intCustomerOrderID = DataEnvironment1.rsSelectCusOrderID.Fields("CustomerOrderID").Value

DataEnvironment1.SelectCusOrderDetail intCustomerOrderID

```

intCountItemID = DataEnvironment1.rsSelectCusOrderDetail.RecordCount
DataEnvironment1.rsSelectCusOrderDetail.MoveFirst

For j = 1 To intCountItemID
    intCusItemID = DataEnvironment1.rsSelectCusOrderDetail.Fields("ItemID").Value
    intQty = DataEnvironment1.rsSelectCusOrderDetail.Fields("OrderQty").Value
    For k = 1 To intNoFinishGoods
        If AddsMasterSchedule(k).intItemID = intCusItemID Then
            AddsMasterSchedule(k).intQty = AddsMasterSchedule(k).intQty + intQty
        End If
    Next
    DataEnvironment1.rsSelectCusOrderDetail.MoveNext
Next
DataEnvironment1.rsSelectCusOrderDetail.Close
DataEnvironment1.rsSelectCusOrderID.MoveNext

Next

For i = 1 To intNoFinishGoods
    DataEnvironment1.CheckMasterSchedule           AddsMasterSchedule(i).intItemID,
AddsMasterSchedule(i).lngDate
    If DataEnvironment1.rsCheckMasterSchedule.EOF Then
        DataEnvironment1.InsertMasterSchedule       AddsMasterSchedule(i).intItemID,
AddsMasterSchedule(i).intQty,                       AddsMasterSchedule(i).strIntervalDate,
AddsMasterSchedule(i).lngDate
    Else
        DataEnvironment1.UpdateMasterSchedule       AddsMasterSchedule(i).intItemID,
AddsMasterSchedule(i).intQty,                       AddsMasterSchedule(i).strIntervalDate,
AddsMasterSchedule(i).lngDate, AddsMasterSchedule(i).intItemID, AddsMasterSchedule(i).lngDate
    End If
    DataEnvironment1.rsCheckMasterSchedule.Close
Next

DataEnvironment1.rsSelectAllFinishGoods.Close

```

endSub:

End Sub

เป็น procedure ที่ใช้ show data ทาง flexgrid

Private Sub ShowData()

Dim icol As Integer

Dim intTest As Integer

DataEnvironment1.ShowMasterSchedule lngDateNew(1)

With flxShowMasterSchedule

DataEnvironment1.rsShowMasterSchedule.MoveFirst

intTest = DataEnvironment1.rsShowMasterSchedule.RecordCount

DataEnvironment1.rsShowMasterSchedule.MoveFirst

Do While Not DataEnvironment1.rsShowMasterSchedule.EOF

.Rows = .Rows + 1

.Row = .Rows - 2

For icol = 0 To DataEnvironment1.rsShowMasterSchedule.Fields.Count - 1

.Col = icol

.Text = DataEnvironment1.rsShowMasterSchedule(icol)

Next

DataEnvironment1.rsShowMasterSchedule.MoveNext

Loop

End With

With flxShowMasterSchedule

.ColWidth(0) = 500

.ColWidth(1) = 1400

.ColWidth(2) = 5400

.FormatString = "ItemID| »ÃÔÁÔ³·Õè¼ÁÔµ |ÇÑ¹·ÕèàÊÃç" "

End With

DataEnvironment1.rsShowMasterSchedule.Close

End Sub

2.4 อธิบายการทำงานของฟอร์ม BOM

The screenshot shows a software window titled "BOM Section". It contains several tables and input fields for managing a Bill of Materials (BOM) for a "Bicycle".

Top Left Table (BOM Header):

รหัสสินค้า	ชื่อสินค้า	เวลานำ
1	Bicycle	1
2	Handle Bars	1

Top Right Section (BOM Detail):

รายการสินค้าสำเร็จรูป

รหัสสินค้า	ชื่อสินค้า	เวลานำ
1	Bicycle	1

Buttons: Delete Finish Goods, Add Finish Goods

Main Section (BOM Structure):

โครงสร้างของสินค้าสำเร็จรูป ID : 1

รหัสสินค้า	ชื่อสินค้า	รหัสสินค้าเบื้องต้น	ระดับ	จำนวนหน่วย	เวลานำ
2	Handle Bars	1	1	1	1
3	Frame Assembly	1	1	1	1
4	Wheels	3	2	1	2
5	Frame	3	2	1	1

Bottom Section (Form Fields):

Fields: รหัสสินค้า, ชื่อสินค้า, รหัสสินค้าเบื้องต้น, ระดับ, จำนวนหน่วย, เวลานำ

Buttons: บันทึกผลงานข้อมูล, **ปรับข้อมูลสินค้าทั้งหมด** (highlighted), Add Item, Delete Item

รูปที่ 2-3 แสดงฟอร์มที่จัดการเกี่ยวกับ BOM

หน้าที่

ตามการใช้งานฟอร์มนี้จะทำหน้าที่จัดการเกี่ยวกับโครงสร้างผลิตภัณฑ์(BOM) ได้แก่ การสร้าง BOM, การแก้ไขและกำหนดโครงสร้างให้แก่ BOM นั้นๆ โดยมีการเรียกใช้ Data Environment DataEn และ Store Procedure

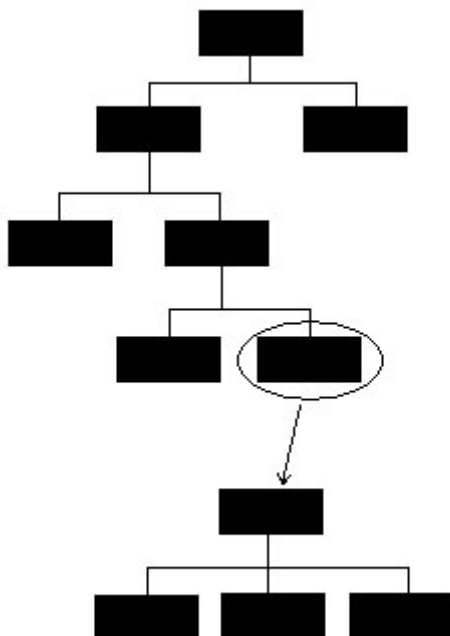
แนวคิด

ในส่วนบนจะเป็นส่วนที่ใช้ในการสร้างตัวที่เป็นสินค้าสำเร็จรูป(เฉพาะตัวบนสุดของแต่ละ tree เท่านั้น) ให้กำหนดรหัสสินค้า ส่วนชื่อกับเวลานำจะกำหนดที่หลังที่หน้า Inventory Detail ก็ได้ แล้วให้กำหนดโครงสร้างต่อในส่วนล่างโดยกำหนดว่ามีชิ้นส่วนอะไรบ้าง อยู่ส่วนไหนของโครงสร้าง โดยกำหนดว่าอยู่ตำแหน่งไหนในโครงสร้างจากการกำหนดตัวชิ้นส่วนเบื้องต้น(Parent)

ในสินค้าสำเร็จรูปนั้นๆ ถ้าหากชิ้นส่วนนั้นเป็นผลิตภัณฑ์สำเร็จรูปก็ไม่ต้องมีชิ้นส่วนที่ต่อจากชิ้นส่วนนั้น ให้ไปกำหนดที่โครงสร้างของผลิตภัณฑ์นั้น เพื่อที่จะได้ทำการคำนวณโครงสร้างในลักษณะ

BOM ซ้อน BOM ได้ถูกต้อง ดังรูปในโครงสร้างของผลิตภัณฑ์อันบนก็จะไม่มีชิ้นส่วนที่เป็นโครงสร้างของผลิตภัณฑ์อันล่าง แล้วตอนทำการคำนวณจะทำการเชื่อมต่อให้เอง

ฐานข้อมูลที่เกี่ยวข้องกับการทำงานในส่วนนี้ ได้แก่ ตาราง BOM, BOM_, InventoryDetail โดย ตาราง BOM_ จะเป็นส่วนที่เก็บโครงสร้างของแต่ละผลิตภัณฑ์ และสามารถทำการหาโครงสร้างที่สมบูรณ์ได้ออกมาเป็นตาราง BOM เพื่อที่จะจัดการกับปัญหาในเรื่องของ BOM ซ้อน BOM ได้



รูปที่ 2-4 แนวคิดเกี่ยวกับการจัดการ BOM ซ้อน BOM

หน้าที่ของแต่ละฟังก์ชัน

- Form_Load()

ทำงานเมื่อฟอร์มทำการโหลด ทำการกำหนดค่าเริ่มต้นของตัวแปร oldSelectRow

ตัวแปร oldSelectRow เป็นตัวแปรประเภท array ขนาด 2 ที่ทำหน้าที่จัดการเกี่ยวกับการทำ Highlight Grid ที่ใช้แสดงผลให้ตัวที่ถูกเลือกเป็นสี ตัวแรกบอกถึง Grid ที่เอาไว้แสดงผลผลิตภัณฑ์สำเร็จรูป ตัวที่สองบอกถึง Grid ที่เอาไว้แสดงโครงสร้างของผลิตภัณฑ์ โดยค่าของตัวแปรจะบอกถึงแถวของ Grid ที่ถูกเลือก เริ่มต้นจะมีค่าเป็น 0 ซึ่งจะไม่แสดงสี

โหลดข้อมูลของ Finish Good แสดงในรายการสินค้าสำเร็จรูปโดยการเรียกฟังก์ชัน loadFG

BOM Section		
รหัสสินค้า	ชื่อสินค้า	เวลาทำ
1	Bicycle	1
2	Handle Bars	1

โครงสร้างของสินค้าสำเร็จรูป

รูปที่ 2-5 ส่วนแสดงรายการผลิตภัณฑ์สำเร็จรูป

- loadFG()

ใช้ data environment getFG ทำการเรียกข้อมูลมาจากฐานข้อมูล เอาข้อมูลที่เป็นผลิตภัณฑ์สำเร็จรูปมาจากตาราง BOM_ ถ้าหากไม่มีก็จะทำการ clear Grid หรือถ้ามีก็จะทำการแสดงข้อมูลออกมา แล้วเรียกฟังก์ชัน initGrid

If DataEn.Conn.State = adStateOpen Then

DataEn.Conn.Close

DataEn.Conn.Open

End If

ในส่วนของโค้ดข้างต้นนั้นจะทำการใช้ทุกครั้งที่ทำการติดต่อกับดาต้าเบสเพื่อตรวจสอบสถานะการติดต่อกับดาต้าเบส มิเช่นนั้นจะ error

- initGrid()

จะเป็นการกำหนดข้อความและขนาดให้กับ Grid เพราะเมื่อมีการโหลดข้อมูลเข้าไปใหม่ จะเป็นค่าเริ่มต้นหมด ต้องทำการกำหนดใหม่

- txtItemID_LostFocus()

ทำการจัดการเมื่อ Focus ได้ออกจาก textbox ที่ทำการกรอกรหัสสินค้าสำเร็จรูป เพื่อทำการตรวจสอบว่ารหัสที่กรอกไปนั้นมีอยู่ในฐานข้อมูลอยู่แล้วหรือไม่ โดยเรียก data environment getItemDes แล้วตรวจสอบว่ามี record ที่ return กลับมามีค่า ถ้าหากเคยมีอยู่แล้วก็จะเอาข้อมูล ชื่อ และ เวลามาอันเก็บมาแสดงให้ดูเพื่อว่าหากต้องการใช้อันเก่าก็ไม่ต้องกรอกใหม่ให้เสียเวลา ถ้าไม่มีก็จะ clear textbox ชื่อและเวลาให้ว่าง

รูปที่ 2-6 ส่วนสำหรับการเพิ่ม ลบ ผลิตภัณฑ์สำเร็จรูป

- cmdAddFG_Click()

ทำงานเมื่อทำการ click ปุ่มเพื่อที่จะเพิ่มรายการผลิตภัณฑ์สำเร็จรูป โดยจะทำการตรวจสอบข้อมูลที่กรอกว่าถูกต้องหรือไม่ก่อน โดยรหัสสินค้าต้องไม่มีค่าเป็นศูนย์ เนื่องจากสงวนไว้เป็นรหัสเฉพาะที่ใช้อ้างอิงถึง parent ของผลิตภัณฑ์สำเร็จรูป ต้องไม่มีค่าเป็นช่องว่าง และไม่เป็นค่าที่เคยมีอยู่แล้ว ถ้าหากตรงตามเงื่อนไขที่จะสามารถเพิ่มได้แล้วก็ทำการเพิ่มข้อมูลลงฐานข้อมูลโดยใช้ data environment dbo_addFG (data environment ที่มีชื่อขึ้นต้นด้วย dbo_ จะเป็น store procedure) แล้วก็ทำการ clear textbox

- cmdDeleteFG_Click()

ทำงานเมื่อทำการ click ปุ่มเพื่อที่จะลบรายการผลิตภัณฑ์สำเร็จรูป โดยจะต้องทำการเลือกก่อนที่จะทำการลบ ก็จะตรวจสอบว่าตัวที่เลือกนั้นมีรหัสสินค้าเป็นช่องว่างหรือเปล่า เพราะว่าถ้ามีค่าเป็นช่องว่างจะหมายถึงยังไม่ได้เลือก หรือไม่มีข้อมูลที่สามารถลบได้ แล้วก็จะแสดง message box บอกก่อนว่าตัวที่กำลังจะลบนั้นเป็นตัวไหน ถ้าตกลงก็จะทำการเรียก data environment deleteFG ทำการลบข้อมูลออกแล้วทำการแสดงผล Grid ใหม่

- mfgBOM1_Click()

เมื่อทำการเลือกที่ Grid ด้านบน จะแสดงข้อมูลของผลิตภัณฑ์ที่ถูกเลือกที่ช่องด้านขวาที่เอามาไว้กรอกสำหรับเพิ่มข้อมูล แล้วทำการจัดการเกี่ยวกับการแสดงสีของแถวที่ถูกเลือก ในการจัดการกับสี จะมีตัวแปรดังที่กล่าวไปแล้วคอยเก็บแถวเก่าที่เคยถูกเลือกไป ก็จะทำการเปลี่ยนสีของแถวที่เคยถูกเลือกให้เป็นสีปกติ แล้วก็เปลี่ยนสีของแถวที่ถูกเลือกให้เป็นสีตามที่ต้องการ แล้วก็ทำการเก็บค่าใหม่ โดยจะมีข้อยกเว้นในกรณีที่เป็นการเลือกครั้งแรก จะไม่ต้องทำการเปลี่ยนค่าแถวเก่าเพราะว่ามีค่าเป็นศูนย์

- mfgBOM1_DblClick()

เมื่อทำการ double click ที่ Grid ด้านบน ที่แถวที่ต้องการให้แสดงโครงสร้าง ก็จะทำการดึงข้อมูลโครงสร้างของผลิตภัณฑ์นั้นมาแสดงที่ Grid ด้านล่าง โดยทำการเรียกฟังก์ชัน loadBOM และจะมีการกำหนดค่าให้ตัวแปร selectBOM ที่บอกว่าได้เลือก BOM อันไหนไป และกำหนด label ให้แสดงว่ากำลังแสดงโครงสร้างของผลิตภัณฑ์ไหนอยู่ตามรูป

รหัสสินค้า	ชื่อสินค้า	เวลาทำ
1	Bicycle	1
2	Handle Bars	1

รหัสสินค้า

Dele

โครงสร้างของสินค้าสำเร็จรูป ID : 1

รหัสสินค้า	ชื่อสินค้า	รหัสสินค้าเบื้องต้น
2	Handle Bars	
3	Frame Assembly	
4	Wheels	

รูปที่ 2-7 ส่วนที่บอกว่ากำลังแสดงโครงสร้างของผลิตภัณฑ์ตัวใดอยู่

- loadBOM()

ทำการดึงข้อมูลที่เป็นโครงสร้างของผลิตภัณฑ์ที่ถูกเลือกนำมาแสดงใน Grid ด้านล่าง โดยใช้ data environment getChild เลือก row ที่มีค่า BOMID เป็นตัวที่ถูกเลือก แต่ว่าไม่ต้องเอาตัวบนสุดมาด้วย ถ้าหากไม่มีค่า return กลับมาซึ่งหมายถึงยังไม่มีโครงสร้าง ก็ทำการ clear grid ให้ว่าง

- txtItemID__GotFocus()

เมื่อช่องสำหรับกรอกรหัสได้ถูกโฟกัสจะทำให้ชื่อและเวลาน่าสามารถที่จะกรอกได้ เพราะว่าจะมีในกรณีที่รหัสที่กรอกถ้ามีอยู่แล้ว จะทำให้กำหนดให้ 2 ช่องนั้นกรอกไม่ได้ให้ไปแก้ไขในหน้า InventoryDetail เพื่อป้องกันความผิดพลาดเนื่องจากชิ้นส่วนอันนี้จะใช้ได้หลายผลิตภัณฑ์ ถ้าหากแก้ไขจะทำให้ในผลิตภัณฑ์อื่นผิดพลาดก็ได้และเพื่อความรวดเร็วในการกรอกข้อมูลด้วย

- txtItemID__LostFocus()

หลังจากช่องสำหรับกรอกรหัสได้หลุดโฟกัสไปแล้ว จะทำการตรวจสอบดูว่ารหัสที่ได้กรอกไปนั้น มีอยู่แล้วในฐานข้อมูลหรือไม่ ถ้าหากมีก็จะนำข้อมูลมาแสดง แล้วไม่ให้สามารถแก้ไขชื่อและเวลาน่าได้ เนื่องจากจะเท่ากันอยู่แล้ว(หากต้องการแก้ไขให้ไปแก้ที่หน้า InventoryDetail)

- cmdAddItem_Click()

จะทำงานเมื่อปุ่มสำหรับเพิ่มชิ้นส่วนให้กับโครงสร้างของผลิตภัณฑ์ถูกกด หากข้อมูลที่กรอกไม่ถูกต้องตามรูปแบบ หรือกรอกไม่ครบก็จะมีข้อความแจ้งเตือน มีการตรวจสอบว่ารหัสสินค้าเบื้องต้นไม่เป็นผลิตภัณฑ์สำหรับรูปเพราะนั่นหมายถึงเป็นการกรอกโครงสร้างของตัวที่เป็นผลิตภัณฑ์สำเร็จรูปซึ่งเป็นชิ้นส่วน ซึ่งคงที่ไปแล้ว หากต้องการทำเช่นนั้นให้ไปแก้ที่ชิ้นส่วนนั้นโดยการดับเบิ้ลคลิกผลิตภัณฑ์สำเร็จรูปนั้นที่ Grid ด้านบนแล้วทำการแก้ไข(หากต้องการทำระหว่างที่กรอกอันเดิมอยู่ให้ทำการบันทึกก่อน)

รูปที่ 2-8 ปุ่มเพิ่มและลบชิ้นส่วนที่เป็นโครงสร้างของผลิตภัณฑ์สำเร็จรูปที่ได้เลือก

มีการตรวจสอบว่าไม่ให้รหัสชิ้นส่วนกับรหัสเบืองบนเหมือนกัน และข้อมูลต้องไม่ซ้ำกับที่เคยกรอกไปแล้วจะถามก่อนว่าต้องการทับอันเดิมหรือไม่ ถ้าตกลงก็จะทำการแก้ไขทับอันเดิม แล้วทำการเคลียร์ช่องที่ได้กรอกไปให้ว่างสำหรับการกรอกชิ้นส่วนต่อไป

ช่องที่บอกเลเวลนั้นจะไม่สามารถกรอกได้เพราะจะทำการหาให้เองเมื่อทำการบันทึก

- cmdDeleteItem_Click()

จะทำงานเมื่อปุ่มสำหรับลบชิ้นส่วนให้กับ โครงสร้างของผลิตภัณฑ์ถูกกด ก็ทำการลบออกจาก Grid โดยจะมีการตรวจสอบก่อนว่าชิ้นส่วนได้ถูกเลือกแล้วรึยัง

ทั้งการเพิ่มและการลบชิ้นส่วนจะเพิ่มและลบจาก Grid เท่านั้น ยังไม่ได้ทำการบันทึกแก้ไขข้อมูลในฐานข้อมูล จะต้องทำการกดปุ่มสำหรับบันทึกก่อน

- mfgBOM2_Click()

ทำหน้าที่เหมือนกัน mfgBOM1_Click() คือจัดการเกี่ยวกับการเปลี่ยนสีแถวของ Grid ที่ถูกเลือก แต่สำหรับ Grid ด้านล่าง

รูปที่ 2-9 ปุ่มบันทึกข้อมูลและปุ่มปรับปรุงข้อมูลผลิตภัณฑ์ที่สัมพันธ์กัน

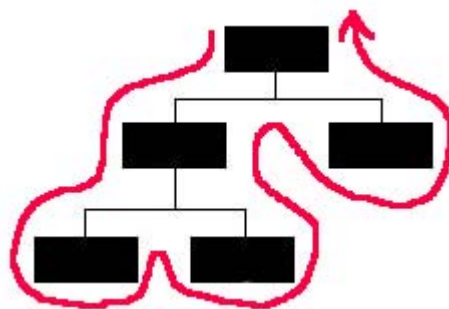
- cmdRecord_Click(), getBOM(ByVal level As Integer, ByVal parentID As Integer, ByRef countItem As Integer, ByVal bomLine As String)

ทำงานเมื่อกดปุ่มบันทึกลงฐานข้อมูล ทำหน้าที่บันทึกโครงสร้างของผลิตภัณฑ์ลงฐานข้อมูล โดยทำงานคู่กับฟังก์ชัน getBOM ในลักษณะของ Recursive ที่ฟังก์ชัน getBOM จะทำการไล่ชิ้นส่วนแต่ละชิ้นไปตาม tree โดยก่อนอื่นจะลบข้อมูลเดิมของ BOM นั้นออกจากตารางในฐานข้อมูลก่อน

มีตัวแปรที่กำหนดขึ้นมาเพื่อทำการหาค่าที่ต้องการ เช่น runLevel เป็นตัวที่ส่งค่าไปด้วยเพื่อบอกว่าชั้นที่ Recursive ไปนั้นอยู่ใน Level ไหน เมื่อลึกลงไปใน tree ก็จะบวกค่าก่อนแล้วส่งค่าต่อไป, ตัวแปร countItem จะทำการนับชิ้นส่วนทุกตัว จะมีการส่งค่าแบบ ByRef แต่ละตัวจะมีค่าไม่ซ้ำกัน

ในแต่ละขั้นของการ Recursive ก็จะมีการตรวจสอบว่าใน Grid ที่เป็นโครงสร้างนั้น มีตัวไหนที่เป็นชิ้นส่วนของตัวที่กำลังอ้างอิงอยู่อีกหรือไม่ ถ้ามีก็จะไล่ไปเรื่อยๆ ถ้าไม่มีก็จะย้อนกลับขึ้นไปยังเลเวลก่อนตัวถัดไปเรื่อยๆจนหมด

ในฟังก์ชัน getBOM จะมีส่วนที่ทำการตรวจสอบด้วยว่าตัวที่กำลังไล่ถึงนั้นมีอยู่แล้วในสายของ tree เดียวกันหรือไม่ เพราะว่าจะไม่ถูกต้อง จะเป็นในลักษณะที่วนไปเรื่อยๆไม่รู้จบ ตัวที่ไม่ถูกต้องตามหลักการนั้นก็จะไม่ถูกบันทึกลงฐานข้อมูล มีข้อความขึ้นแจ้งเตือนว่าโครงสร้างของ tree ไม่ถูกต้องให้ทำการตรวจสอบ



รูปที่ 2-10 การไล่ชิ้นส่วนในการ Recursive

- cmdRelation_Click()

ทำงานเมื่อปุ่มปรับข้อมูลสินค้าทั้งหมดถูกกด โดยจะมีหน้าที่ทำการปรับปรุงข้อมูลของผลิตภัณฑ์ในตาราง BOM โดยดูจากความสัมพันธ์ของผลิตภัณฑ์ทั้งหมด อย่างกรณี BOM ซ้อน BOM ในส่วนนี้ก็จะทำการเชื่อมผลิตภัณฑ์ให้เห็นโครงสร้างของผลิตภัณฑ์สำเร็จรูปนั้นทั้งหมด โดยการเรียกฟังก์ชัน createBOM ทำงานกับฟังก์ชัน getBOM_ จะทำงานเหมือนกับ การบันทึกข้อมูลลงฐานข้อมูลดังที่ได้กล่าวไปแล้ว

มีข้อแตกต่างกับการบันทึกข้อมูลตรงที่ ส่วนท้ายของ getBOM_ จะมองต่อไปอีกด้วยว่าที่ BOM นั้นต่อจากชิ้นส่วนที่กำลังไล่อยู่หรือไม่ ระหว่างที่ทำการไล่ก็จะคำนวณหาค่า Critical Leadtime ของแต่ละส่วนไปด้วย หลังจากทำการไล่เสร็จทั้งหมดก็จะทำการหาค่า Low Level ของชิ้นส่วนทั้งหมด เพราะว่าค่าได้มีการเปลี่ยนแปลงต้องหาใหม่

2.5 อธิบายการทำงานของฟอร์ม Print



รูปที่ 2-11 ฟอร์มที่สั่งให้ออกรายงาน

เมื่อฟอร์มโหลดขึ้นมาก็จะกำหนดค่าให้กับ Combo box อันบนว่ามีรายงานอะไรบ้างที่สามารถออกได้

เมื่อทำการเลือกการจะเข้าไปทำงานในฟังก์ชัน Combo1_Click() โดยจะดูว่าได้เลือกค่าอะไรไปแล้วแยกทำตามเงื่อนไข ถ้าหากรายงานนั้นต้องการส่งค่าไปให้ด้วย ก็จะต้องสั่งให้ Combo box อีกอัน visible แล้วดึงค่าออกมาจากฐานข้อมูลให้เลือกว่าต้องการที่ดูรายงานที่มีการกำหนดค่าเป็นอะไร ตามแต่หัวข้อรายงานที่ต้องการออก ถ้าหากไม่มีการส่งค่าก็ไม่ต้องให้ Combo box อันล่าง visible

หลังจากเลือกได้ตามต้องการแล้ว หากกดปุ่ม View Report ก็จะทำงานฟังก์ชัน Command1_Click() การกำหนดตามค่าที่ได้เลือกไปว่าจะกำหนดให้ดูรูปแบบจากไฟล์ report อันไหน แล้วสั่งให้ preview report ก่อนที่จะทำการพิมพ์ทุกครั้ง หากอันไหนมีการส่งค่าก็จะทำการกำหนดค่าในส่วนนี้ด้วย